

Probability and Statistics for Genomic Data Science

DATA405 Genomic Data Science • Lecture 11 • Sep 29, 2026

ATGCCGGAGGCCCCGCCTCTGCTGTTGGCAGCTGTGTTGCTGGGCCTGGTGCTGCTGGTGGTG

Jinrui Xu, Department of Biology, Howard University

Sequencing is sampling: the ideas behind DESeq2, EM/RSEM and ChIP-seq peak calling

Where we are

- **Yesterday:** Python and R — strings, tables, a t-test on SPARCL1 with $p = 0.000003$.
- **The question we left open:** why is that t-test the wrong tool for 19,271 genes and 4 donors?
- **Today:** the three distributions of genomics, estimation (likelihood, Bayes, EM), and how DESeq2, RSEM and peak callers use them.
- **Tomorrow:** Lab 1 — you will run the binomial test, the EM loop and the BH correction yourself.

Today in one sentence

A sequencer does not measure expression or binding. It **draws reads at random** from a pool of molecules.

Every number in genomics is a count from a random sample.

Every claim therefore needs a probability model — and the model decides what counts as a discovery.

Three distributions, one estimation idea, one correction. That is most of applied genomic statistics.

The plan

Part	Question	Tool
1 Sequencing is sampling	How do reads land on a site, a window, a gene?	Binomial · Poisson · Negative binomial
2 Estimation	Given counts, what is the parameter?	Likelihood · Bayes (Beta) · EM
3 EM in RNA-seq	Which isoform did a shared read come from?	RSEM (your BIOL419 slides)
4 Differential expression	Which genes change, and how sure are we?	Size factors · NB GLM · BH — DESeq2
5 ChIP-seq	Is this peak real? Which control?	Poisson peaks · mock IP vs input · Beta posterior
6 Toolbox	Which tool for which question?	One table

Part 1

Sequencing is sampling

Binomial for alleles, Poisson for windows, negative binomial for replicates.

Three distributions cover most of genomics

- **Binomial(n, θ)**: of n reads at a site, how many carry allele A? — genotyping, allele-specific expression, RNA editing.
- **Poisson(λ)**: how many reads land in a window or on a gene? — coverage (Lander-Waterman, L6), ChIP-seq peaks (MACS).
- **Negative binomial(μ, α)**: the same count across biological replicates — RNA-seq differential expression (DESeq2, edgeR).
- Each is the previous one plus one more source of randomness.

Binomial $\rightarrow$ Poisson is the same coin flipped many times with small p . Poisson $\rightarrow$ NB adds a random λ per replicate.

Binomial: is this mismatch a heterozygous SNP?

If the sample is heterozygous, each read carries the alt allele with $\theta = 0.5$. How surprising is what we saw?

Python

```
from scipy import stats
# 54 alt reads of 138 (lab data)
stats.binomtest(54, 138, 0.5)
# p = 0.013, estimate 0.39

# 9 of 9 reads carry the alt
stats.binomtest(9, 9, 0.5).pvalue
# p = 0.0039
```

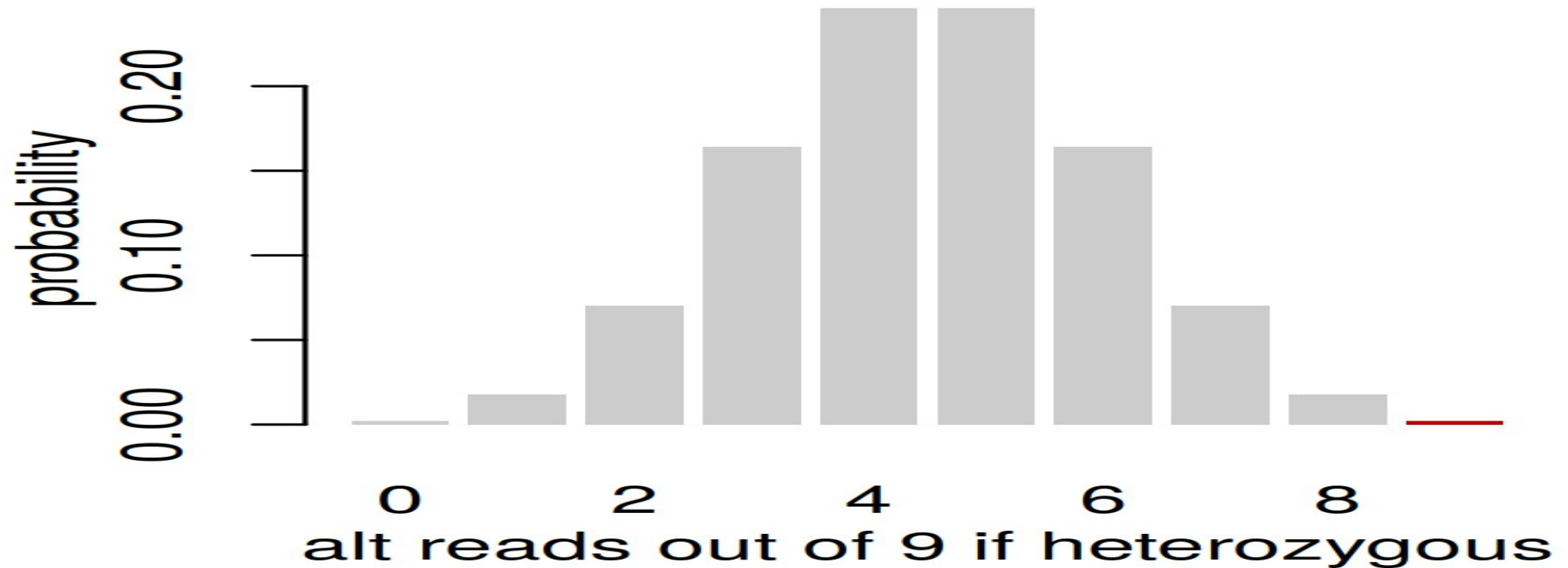
R

```
# 54 alt reads of 138 (lab data)
binom.test(54, 138, 0.5)
# p = 0.013, estimate 0.39
# 95% CI 0.31 - 0.48

# 9 of 9 reads carry the alt
binom.test(9, 9, 0.5)$p.value
# p = 0.0039
```

54/138 is a little low for 0.5 (allelic imbalance? mapping bias?). 9/9 is not heterozygous at all — a homozygous difference, or mis-mapped reads.

Binomial(9, 0.5): the chance of 9 out of 9



$$P(X = 9 \mid n = 9, \theta = 0.5) = 1/512 \approx 0.002 \text{ (two-sided } p = 0.0039)$$

Most all-alt sites in your lab table are not SNPs in this person; they are reads from a paralog mapping to chr22.

Poisson: why 25 reads in one window is a peak

If reads fell uniformly, a window that averages $\lambda = 5$ reads would receive 25 or more with probability...

Python

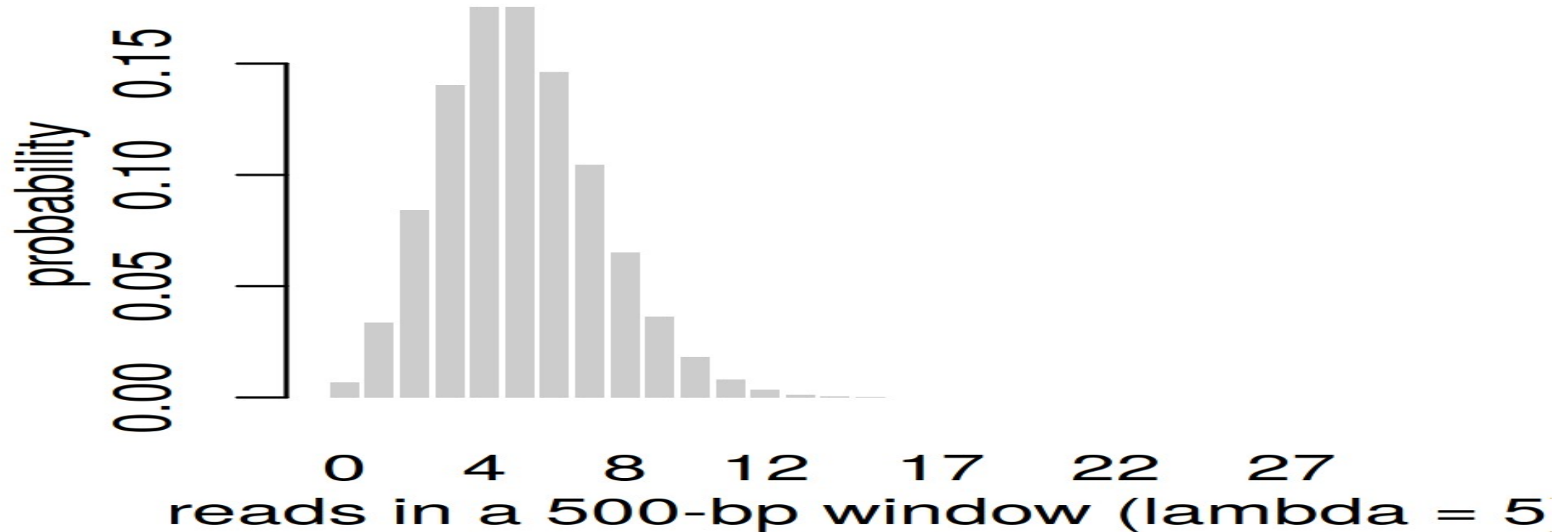
```
# background: 5 reads per 500-bp window
stats.poisson.sf(24, 5)
# P(X >= 25 | lambda = 5) = 1.6e-10
```

R

```
# background: 5 reads per 500-bp window
ppois(24, 5, lower.tail = FALSE)
# P(X >= 25 | lambda = 5) = 1.6e-10
```

This IS a peak caller. MACS computes a local λ from the control and asks this question at every window. The hard part is λ (Part 5).

Poisson($\lambda = 5$): the tail is where discoveries live



Red bars: 25 or more reads. Their total probability is 1.6×10^{-10} .

One assumption hides here: reads fall uniformly. Open chromatin, GC content and mappability all break it — hence controls.

Replicates: Poisson is not enough

- Poisson has one parameter: mean = variance. Technical replicates (same library sequenced twice) obey it.
- Biological replicates (four donors) do not: each donor has its own true expression, so the count's mean itself varies.
- **Negative binomial:** variance = $\mu + \alpha \cdot \mu^2$. The extra term is the dispersion α — the biological variability.
- Look at the airway control samples: does the variance track the mean?

Mean vs variance in the four control samples

Python

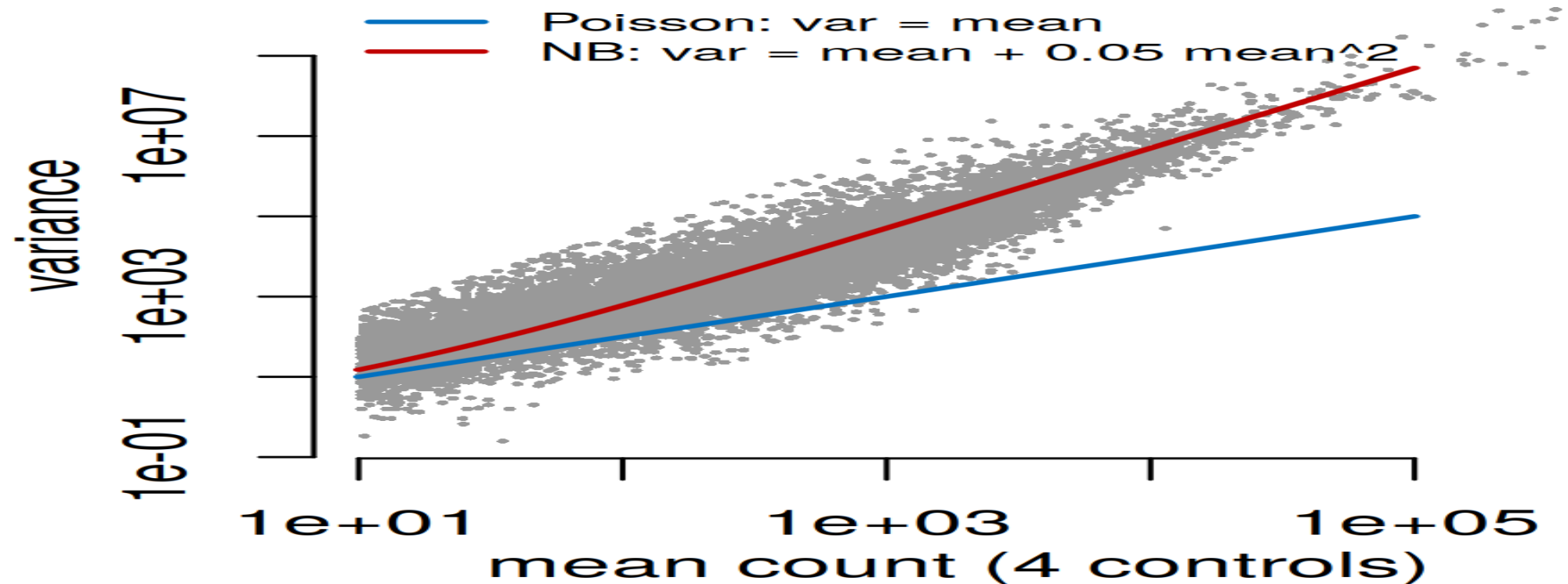
```
cts = pd.read_csv("data/airway_counts.tsv",
                  sep="\t", index_col=0)
meta = pd.read_csv("data/airway_metadata.tsv",
                  sep="\t")
ctrl = list(meta.id[meta.dex == "control"])
mu = cts[ctrl].mean(axis=1)
var = cts[ctrl].var(axis=1)
sel = mu > 10
(var[sel] > mu[sel]).mean()      # 0.95
```

R

```
cts <- read.delim("data/airway_counts.tsv",
                 row.names = 1)
meta <- read.delim("data/airway_metadata.tsv")
ctrl <- meta$id[meta$dex == "control"]
mu <- rowMeans(cts[, ctrl])
v <- apply(cts[, ctrl], 1, var)
sel <- mu > 10
mean(v[sel] > mu[sel])          # 0.95
plot(mu[sel], v[sel], log = "xy")
```

95 % of genes have variance above the Poisson line. The scatter is biology, not sequencing.

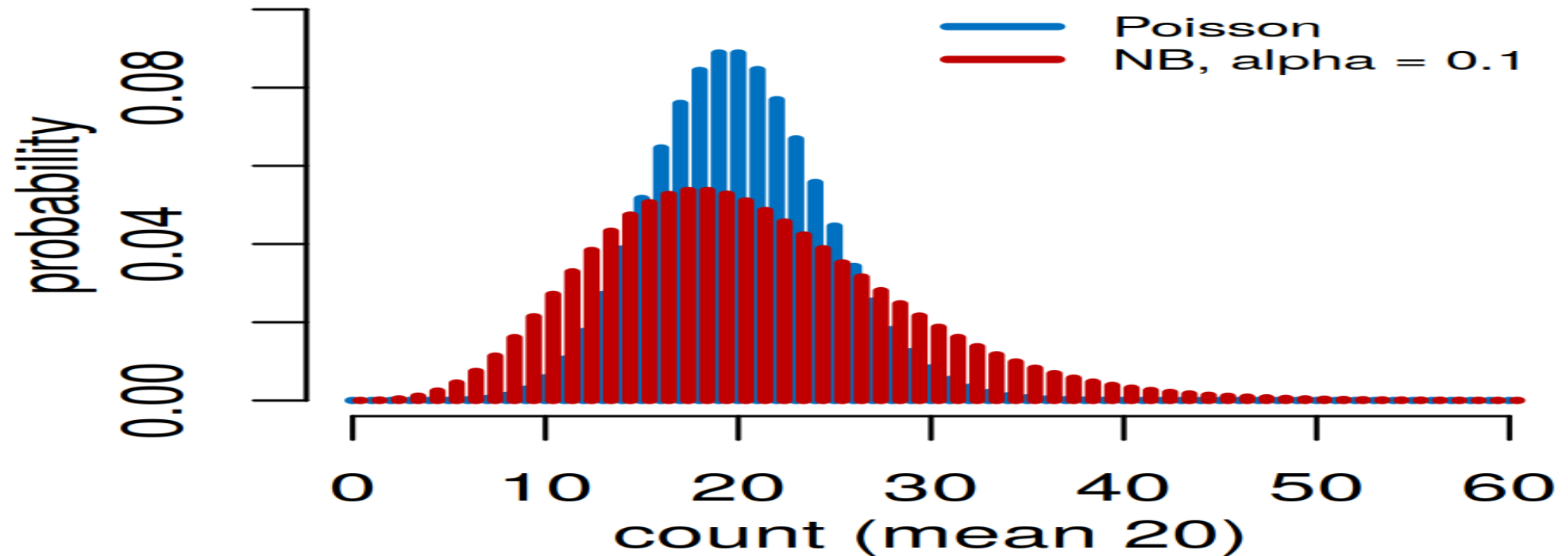
Overdispersion: the negative binomial fits, Poisson does not



Each dot is a gene. Blue: Poisson ($\text{var} = \text{mean}$). Red: NB with $\alpha = 0.05$.

DESeq2's central task: estimate α for every gene from 4 vs 4 samples — which is why it borrows strength across genes.

Same mean, different spread



Mean 20. Poisson: sd 4.5. NB with $\alpha = 0.1$: sd 6.9. A count of 35 is rare under Poisson, ordinary under NB.

Pretend counts are Poisson and every replicate difference looks significant. That is the classic false-positive machine.

Which distribution?

YOUR TURN Match each question to Binomial, Poisson or Negative binomial.

- 1. Of 40 reads covering rs4680, 18 carry A. Heterozygous?
- 2. A 1-kb window received 60 ChIP reads; the input averages 12. Peak?
- 3. COMT has 8, 11, 6, 9 counts in four controls and 30, 25, 41, 28 in four treated. Changed?

ANSWER 1 Binomial ($n = 40$, $\theta = 0.5$) · 2 Poisson ($\lambda = 12$) · 3 Negative binomial — replicates across people.

Part 2

Estimation

Likelihood, Bayes, and the EM idea — what to do when part of the data is hidden.

Maximum likelihood: choose the parameter that makes the data most probable

- 54 alt reads of 138. For each θ , the binomial gives $P(\text{data} \mid \theta)$. The θ that maximizes it is $\hat{\theta} = 54/138 = 0.39$.
- The obvious answer — now with a principle behind it. The same principle gives the sample mean for Poisson and the GLM fit in DESeq2.
- **Likelihood answers:** which parameter fits best? It does not answer: how sure am I that $\theta > 0.5$?

Every estimator in genomics — TPM, fold change, peak height — is a likelihood-type estimate from a sample.

Bayes: turn the likelihood into a probability about the parameter

- Posterior $\propto$ likelihood $\times$ prior. With a flat prior on θ and k alt reads out of n : $\theta \mid \text{data} \sim \text{Beta}(k + 1, n - k + 1)$.
- Now we can ask $P(\theta > 0.5 \mid \text{data})$ directly — a p-value never says this.
- With little data the posterior is wide (9 reads $\rightarrow$ Beta(10, 1) still spreads from 0.7 to 1); with more data it narrows.
- This Beta posterior is exactly what the mock-IP paper uses in Part 5.

Likelihood: ' $\theta = 0.39$ fits best.' Bayes: ' θ is between 0.31 and 0.48 with 95 % probability.'

EM: when part of the data is hidden

- A read compatible with two transcripts has a hidden label: which one did it come from?
- **E-step:** assign hidden labels in expectation, using the current parameters (abundances).
- **M-step:** re-estimate the parameters by maximum likelihood, as if the assignments were data.
- Repeat. Each round can only increase the likelihood; stop when nothing changes.
- Your BIOL419 slides showed this with pies and coloured transcripts. Let us walk through them, then write it in ten lines.

Part 3

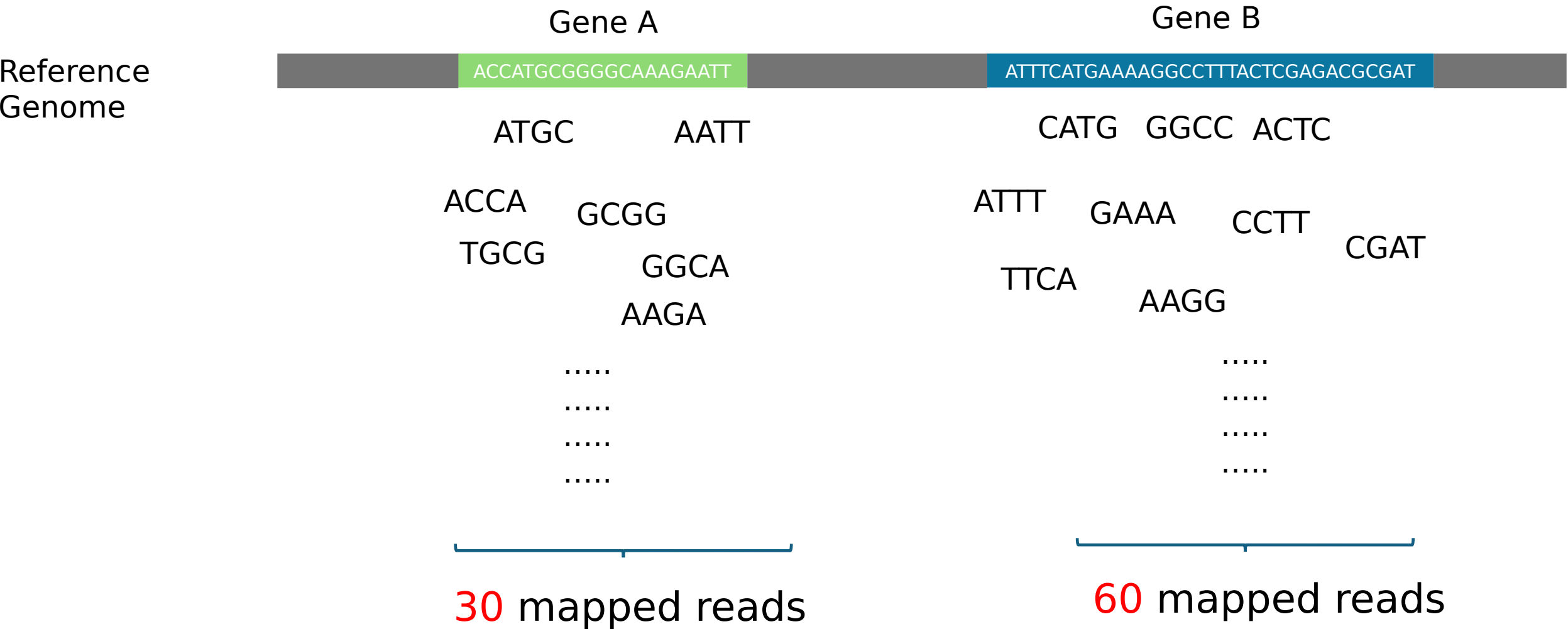
EM for RNA-seq (RSEM)

The multi-mapping problem and the EM solution — from BIOL419 Lecture 12.

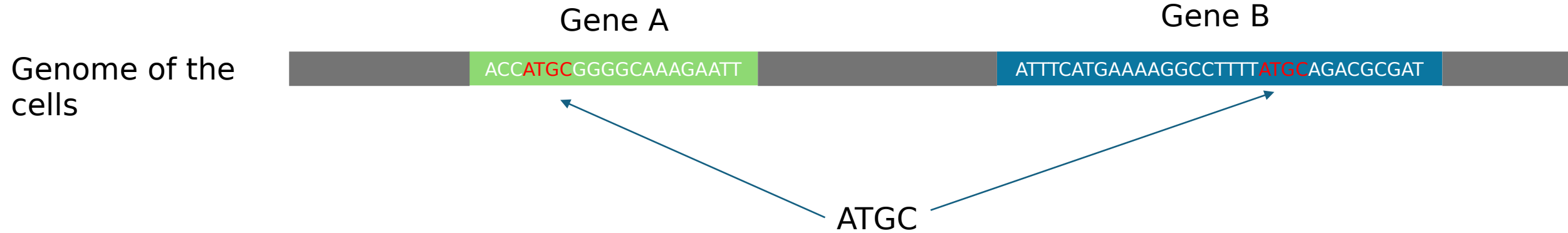
You: *RNA-seq data analysis is conceptually easy!*

Me: *Well... there are some complications.*

Simplified Read Mapping



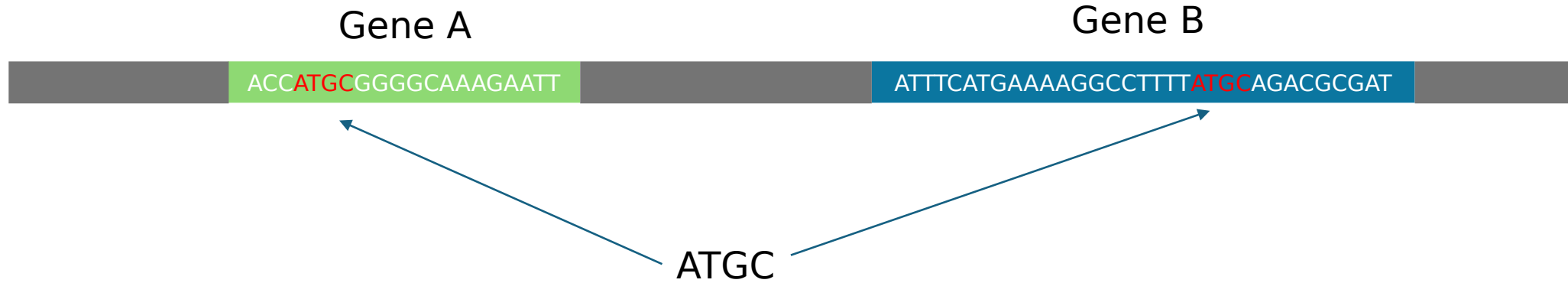
In reality



What should we do for this multi-mapping read?

In reality

Genome of the
cells

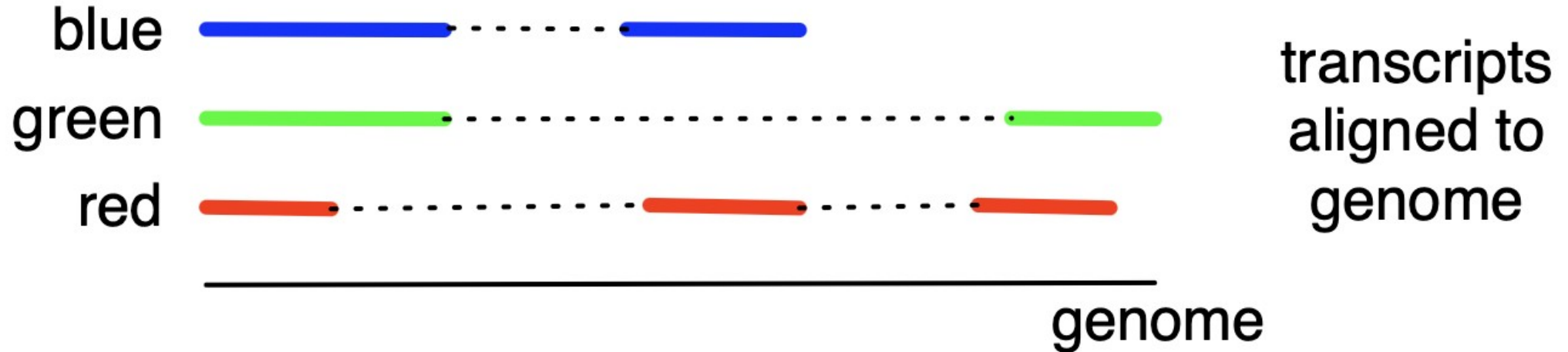


What should we do for this multi-mapping read?

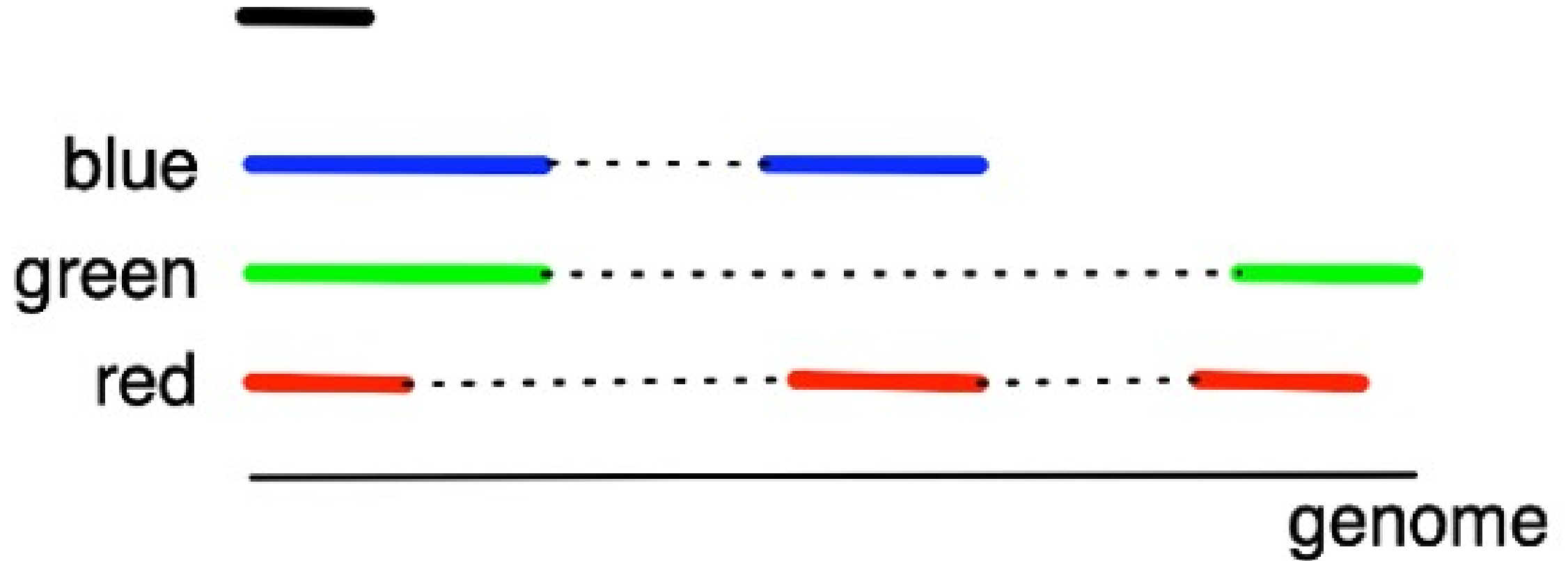
- 1: Drop the read, and pretend we have never seen it
- 2: Count 0.5 for **Gene A** and 0.5 for **Gene B**

A more challenging problem

Each human gene usually have multiple **transcripts**



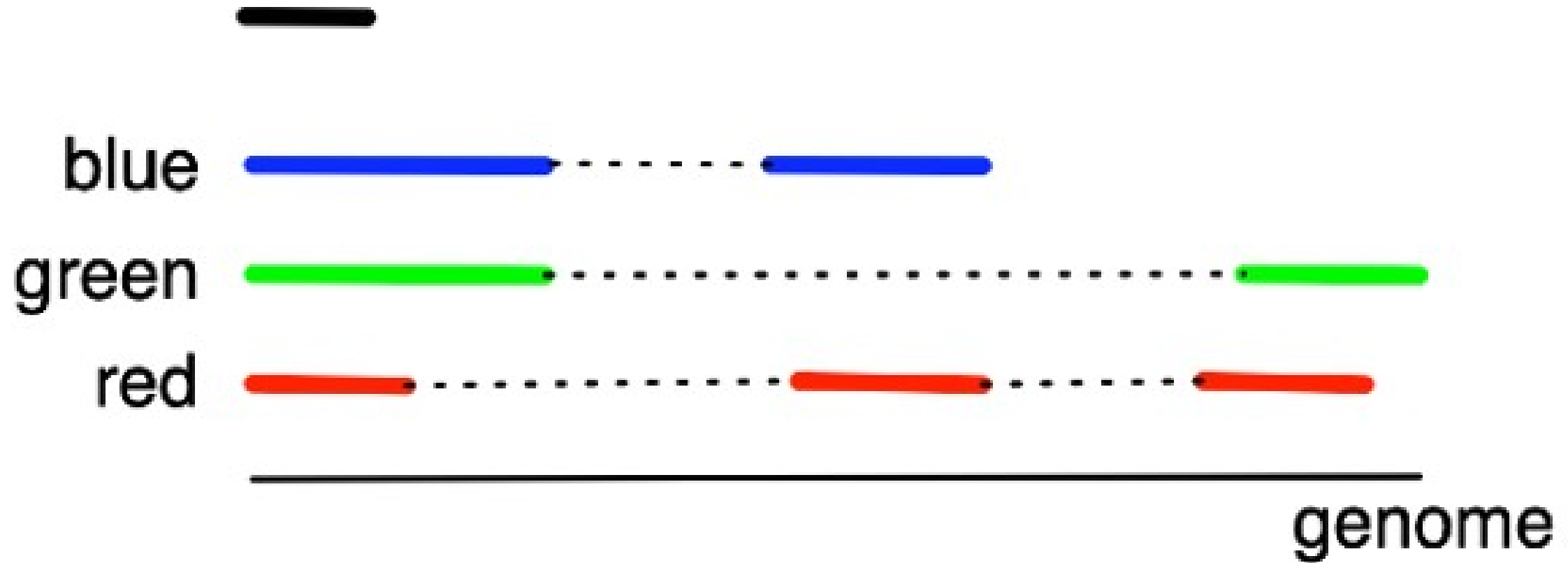
A more challenging problem



A more challenging problem

(1) Drop the read?

(2) Split the read to $1/3$, $1/3$, and $1/3$?

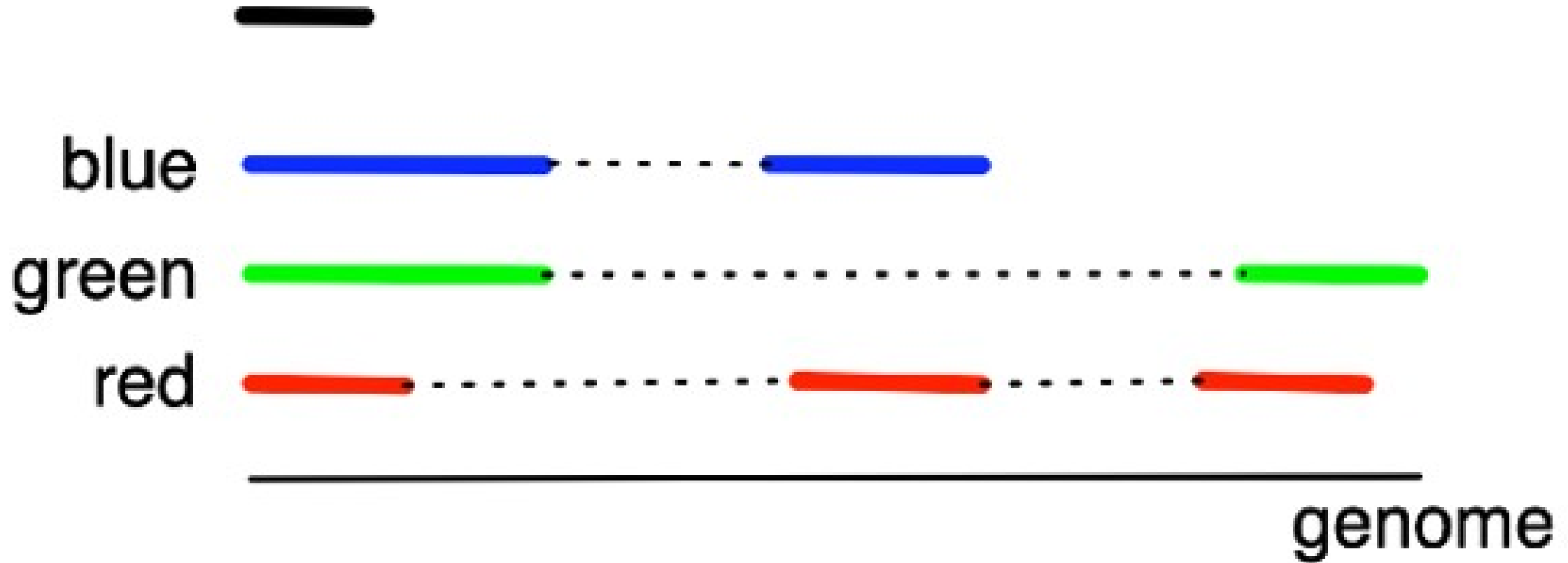


A more challenging problem

(1) Drop the read?

(2) Split the read to $\frac{1}{3}$, $\frac{1}{3}$, and $\frac{1}{3}$?

Or any better solution?



A more challenging problem

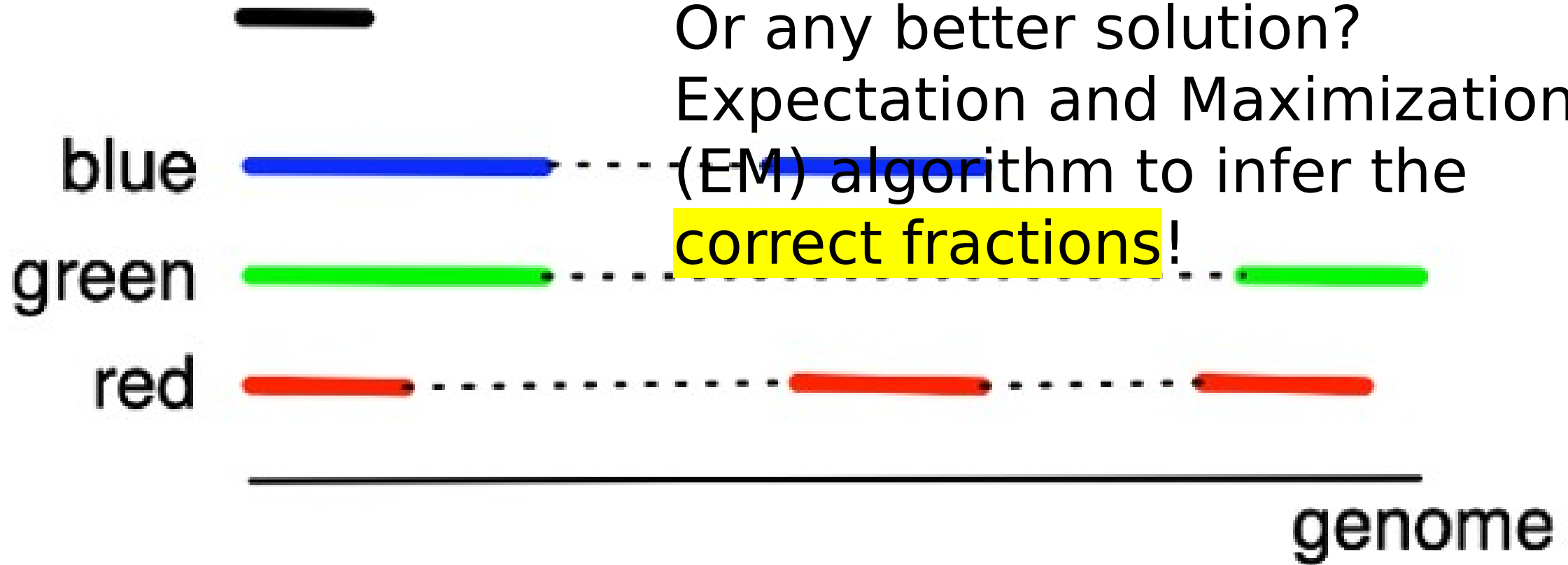
(1) Drop the read?

(2) Split the read to $\frac{1}{3}$, $\frac{1}{3}$, and $\frac{1}{3}$?

Or any better solution?

Expectation and Maximization

(EM) algorithm to infer the
correct fractions!



Expectation-Maximization Algorithm for RNA-Seq

Expectation-Maximization Algorithm

E Step: Compute values of missing information z given an estimate of θ at time t .

$$q^t(z) = P(z|r, \theta^{(t)})$$

M Step: Update the estimates of θ based on new z values that maximize the data likelihood.

$$Q(\theta|\theta^{(t)}) = \sum_z q^t(z) \log[P(r, \hat{z}|\theta)]$$

$$\hat{\theta}^{(t+1)} = \operatorname{argmax}_{\theta} Q(\theta|\theta^{(t)})$$

$$\hat{\theta}^{(t+1)} = \operatorname{argmax}_{\theta} Q(\theta|\theta^{(t)})$$

$$\begin{aligned} \log[P(r|\theta)] &= \log\left(\sum_z P(r, z|\theta)\right) \\ &= \log\left[\sum_z q(z) \cdot \frac{P(r, z|\theta)}{q(z)}\right] = \log\left[E_{qz}\left(\frac{P(r, z|\theta)}{q(z)}\right)\right] \\ &\geq \sum_z q(z) \log\left[\frac{P(r, z|\theta)}{q(z)}\right] = E_{qz}\left[\log\left(\frac{P(r, z|\theta)}{q(z)}\right)\right] \end{aligned} \quad (2.1)$$

$$(2.2)$$

The penultimate step (from Equation 2.1 to 2.2) uses Jensen's inequality to get a lower bound on the log data likelihood, which is the quantity we want to maximize.

Definition 2.1 (Jenson's Inequality) Given a convex function f and random variable X ,

$$E[f(X)] \geq f(EX).$$

If f is strictly convex, then

$$E[f(X)] = f(EX) \iff X \text{ is constant.}$$

Jensen's inequality also holds for concave function f with the inequality reversed:

$$f(EX) \geq E[f(X)]$$

(Andrew 2018).

A strictly concave function is a function f where $f''(x) \leq 0$. In the terms of the EM algorithm, we conclude that $\log\left[E_{qz}\left(\frac{P(r, z|\theta)}{q(z)}\right)\right] \geq E_{qz}\left[\log\left(\frac{P(r, z|\theta)}{q(z)}\right)\right]$ from Equations 2.1 and 2.2. Here, $f(x) = \log(x)$, where $x = \frac{P(r, z|\theta)}{q(z)}$, is a strictly concave function because $f''(x) = -x^{-2} \leq 0$. Thus, $f(EX) \geq E[f(X)]$. Note that the inequality becomes an equality when $\frac{P(r, z|\theta)}{q(z)}$ is equal to a constant.

$$\begin{aligned} c &= \frac{P(r, z|\theta)}{q(z)} \\ q(z) &= \frac{P(r, z|\theta)}{c} \\ &= \frac{P(r, z|\theta)}{\sum_z P(r, z|\theta)} \text{ because } \sum_z q(z) = 1 \\ &= \frac{P(r, z|\theta)}{P(r|\theta)} \\ &= P(z|r, \theta) \end{aligned}$$

By setting $q(z) = P(z|r, \theta)$ (the posterior distribution of z over the data r and the current estimate of θ), we get an equality for the the log likelihood of the data, $\log(P(r|\theta))$. For the true value of θ (unknown), we can say:

$$\log(P(r|\theta)) = \sum_z q(z) \log\left(\frac{P(r, z|\theta)}{q(z)}\right) = \sum_z P(z|r, \theta) \log\left(\frac{P(r, z|\theta)}{P(z|r, \theta)}\right)$$

For the given estimate of θ at time t (θ^t), and the associated $q^t(z) = P(z|r, \theta^t)$ we can say:

$$\log(P(r|\theta^t)) = \sum_z q^t(z) \log\left(\frac{P(r, z|\theta^t)}{q^t(z)}\right) = \sum_z P(z|r, \theta^t) \log\left(\frac{P(r, z|\theta^t)}{P(z|r, \theta^t)}\right)$$

Here, we can get the parameter $\theta^{(t+1)}$ by maximizing the likelihood involving the previous estimate of the parameter. Because in this specific case, $q^t(z)$ is not defined as $P(z|r, \theta)$, but rather $P(z|r, \theta^t)$, the following is true due to Jenson's inequality (note there is no equality due to $q(z) \neq q^t(z)$):

$$\log(P(r|\theta)) \geq \sum_z q^t(z) \log\left(\frac{P(r, z|\theta^t)}{q^t(z)}\right) = \sum_z P(z|r, \theta^t) \log\left(\frac{P(r, z|\theta^t)}{P(z|r, \theta^t)}\right)$$

Further simplifying the log likelihood at a given estimate at time t gives us:

$$\begin{aligned} \log(P(r|\theta^t)) &= \sum_z q^t(z) \log\left(\frac{P(r, z|\theta^t)}{q^t(z)}\right) \\ &= \sum_z q^t(z) \log[P(r, z|\theta^t)] - \sum_z q^t(z) \log(q^t(z)) \end{aligned}$$

Note that the second term is independent of θ . Because we want to maximize the likelihood with respect to θ , we only need to maximize

$$Q(\theta|\theta^{(t)}) = E_{q^t(z)}[\log(P(r, z|\theta^t))] = \sum_z q^t(z) \log(P(r, z|\theta^t))$$

where $\theta^{(t)}$ is the current estimate of θ . We now iterate between the following two steps of the algorithm, where each iteration never decreases the likelihood of the data under the estimated parameters (Dempster, Laird, and Rubin 1977).

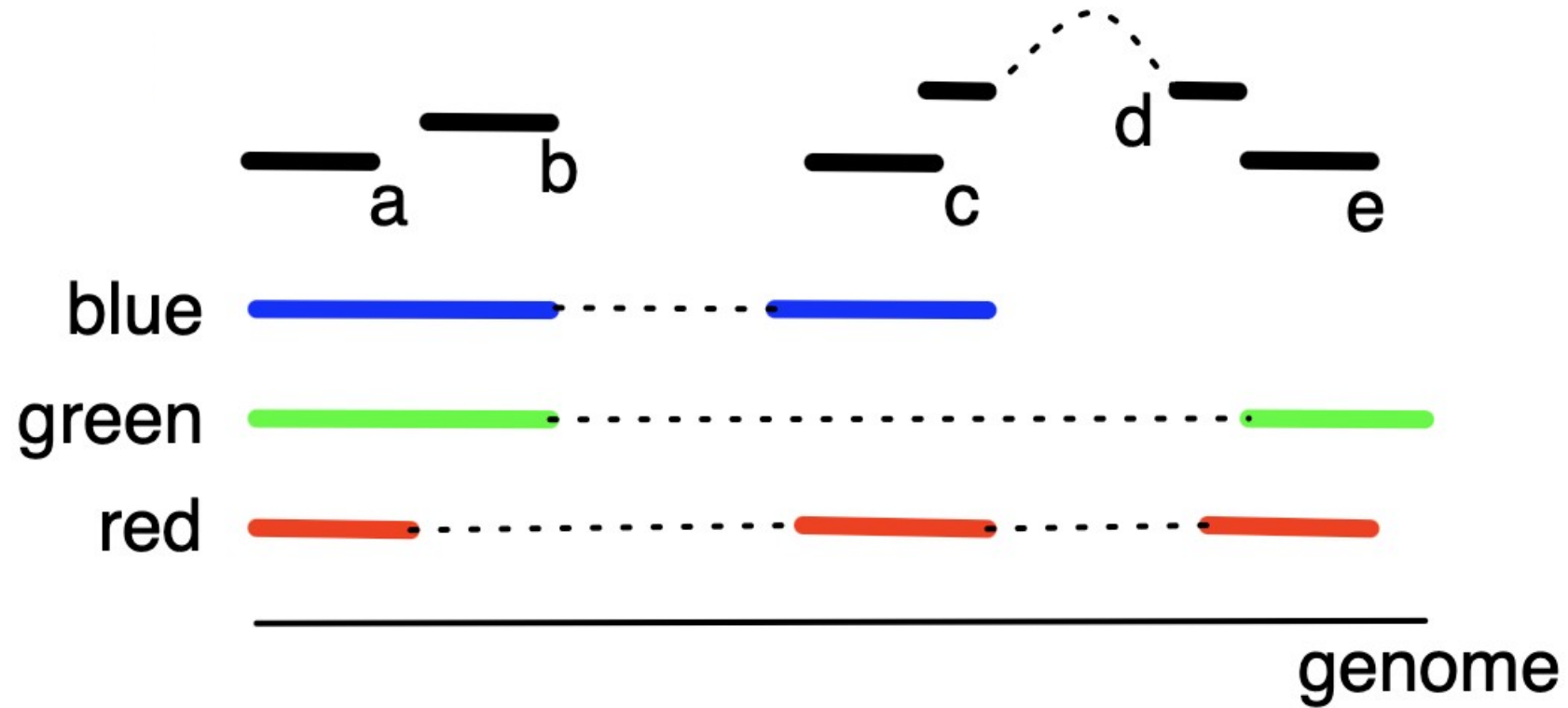
1. E step: $Q(\theta|\theta^{(t)})$ is the expected value of the log likelihood function of θ with respect to the current conditional distribution of z given an estimate of θ and data r .

$$q(z) = P(z|r, \theta^{(t)})$$

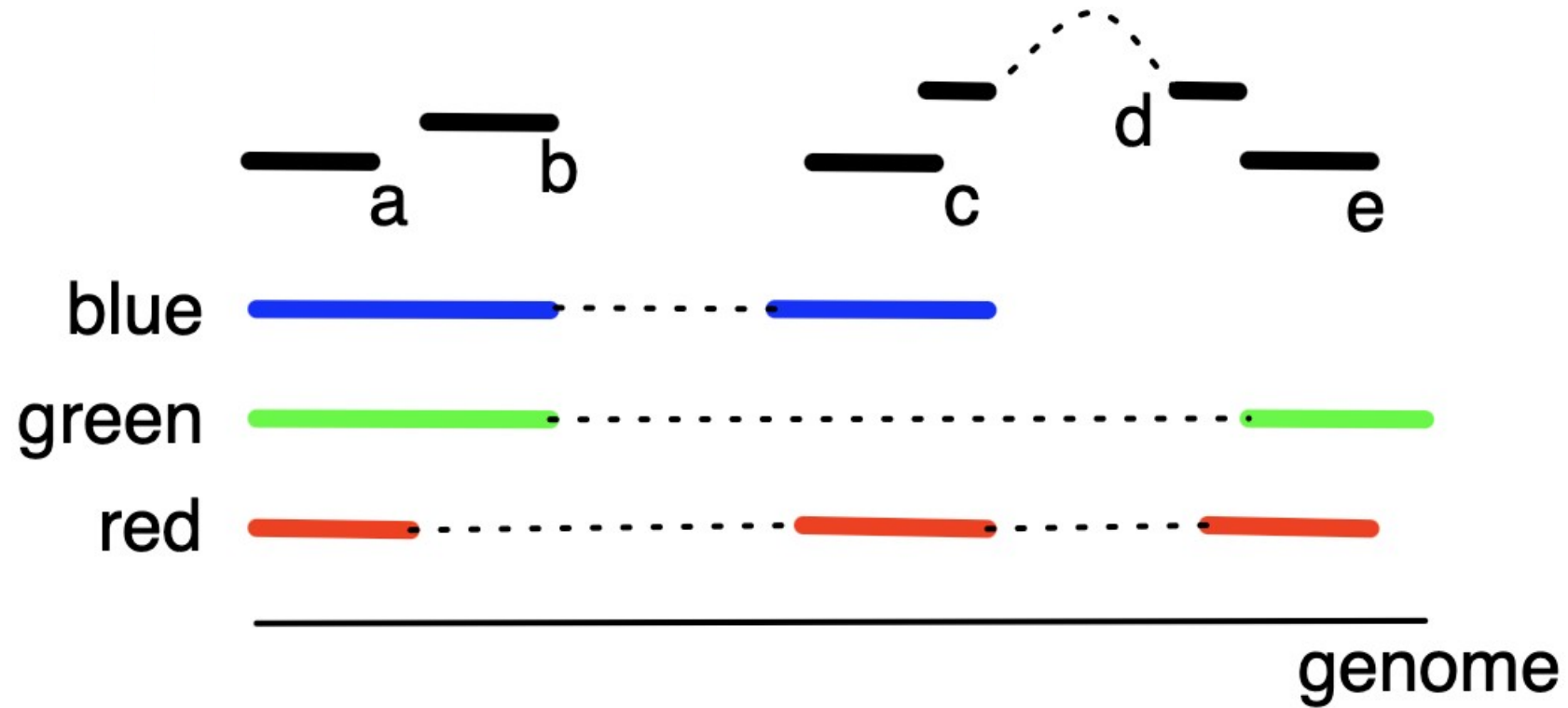
$$Q(\theta|\theta^{(t)}) = \sum_z q(z) \log(P(r, z|\theta))$$

$$Q(\theta|\theta^{(t)}) = \sum_z P(z|r, \theta^{(t)}) \log(P(r, z|\theta))$$

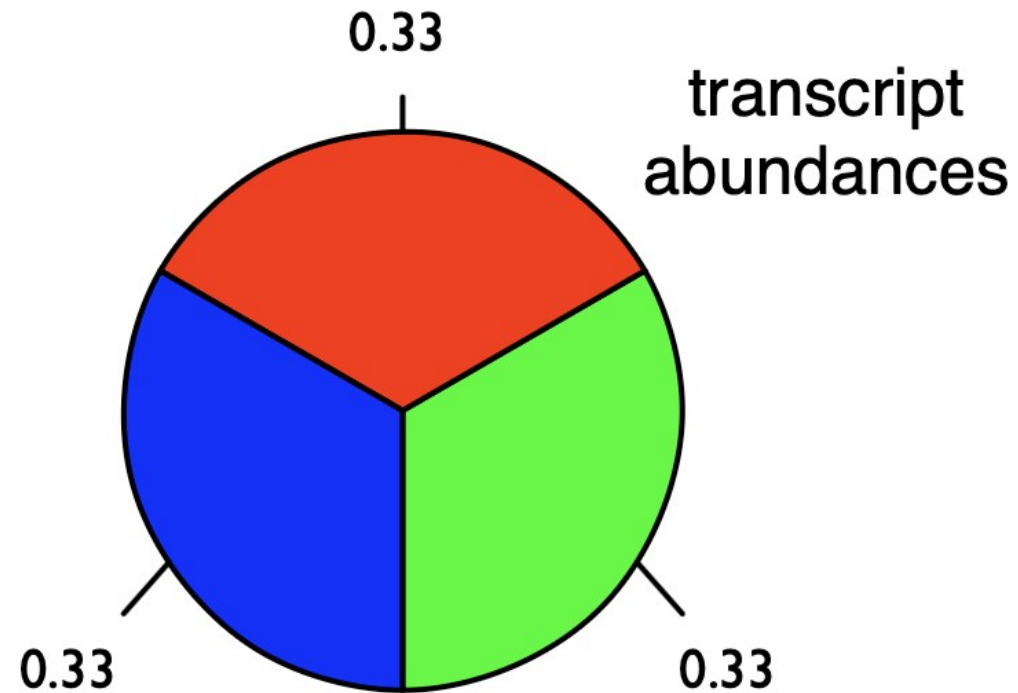
Let's learn by example and use just you intuition!



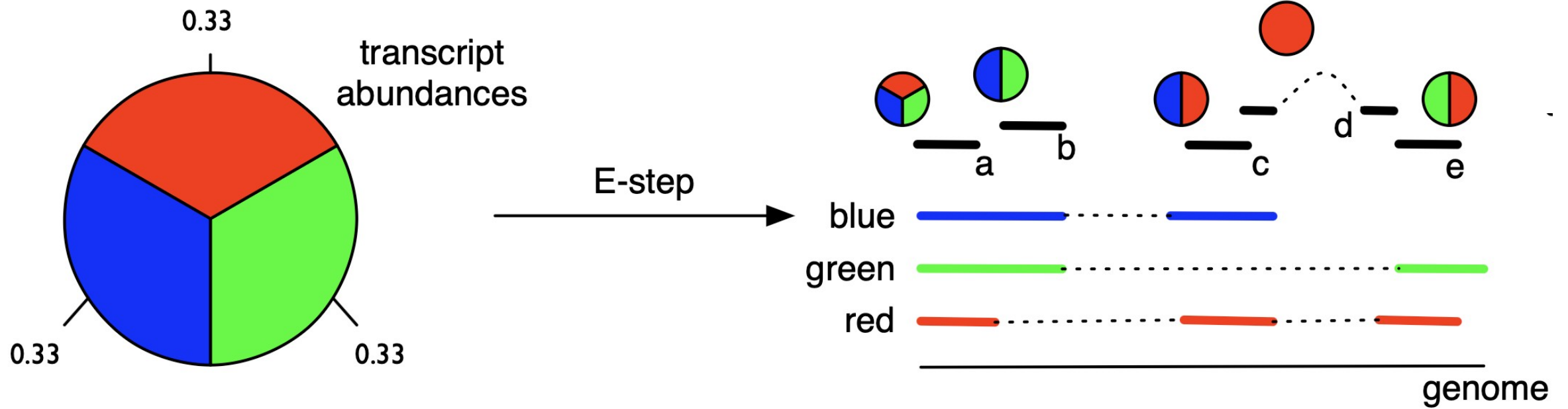
E-M Principle: reads are more likely to be mapped to transcripts with higher abundances



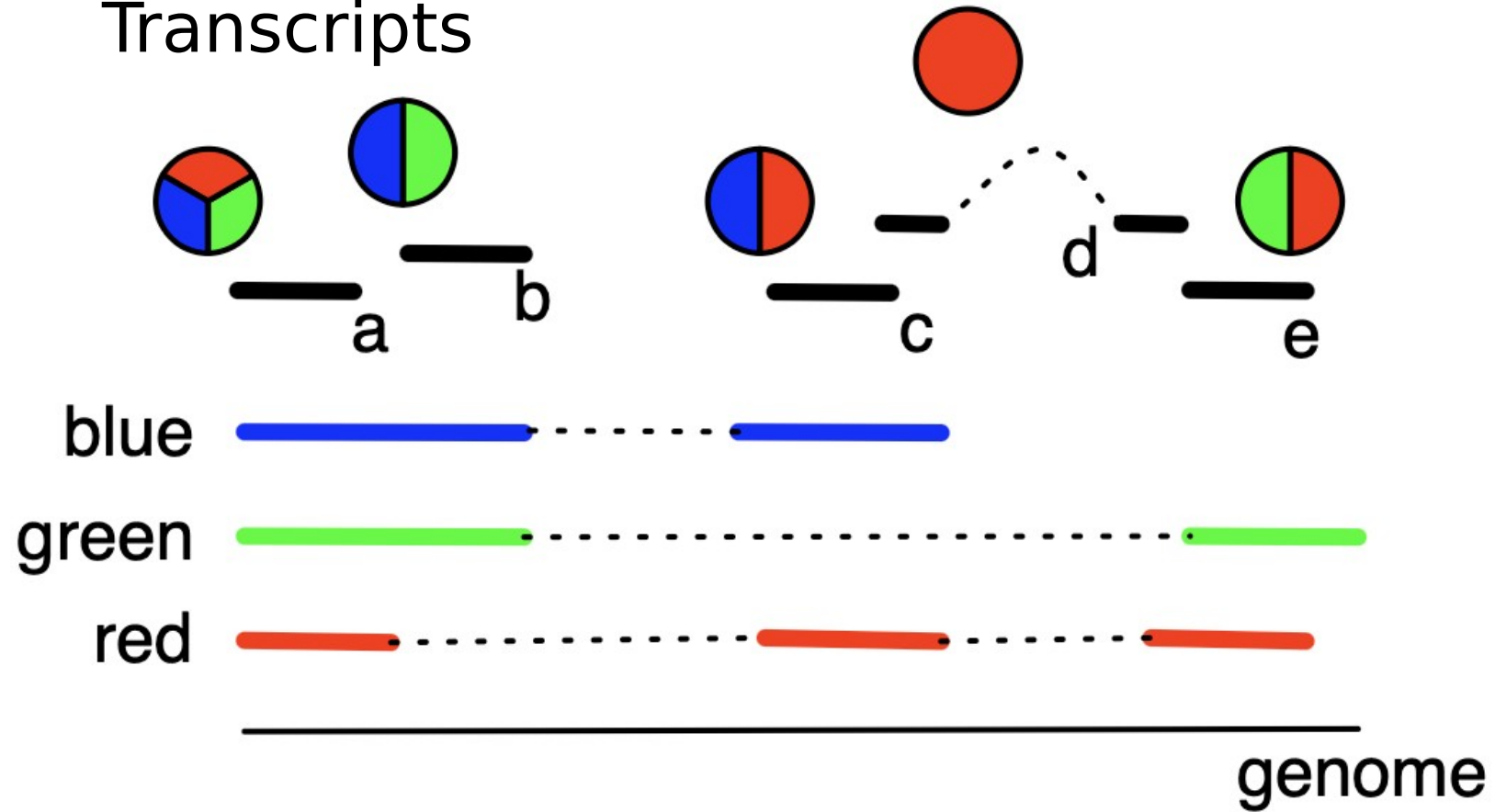
Initiation: All the transcripts have the same abundances



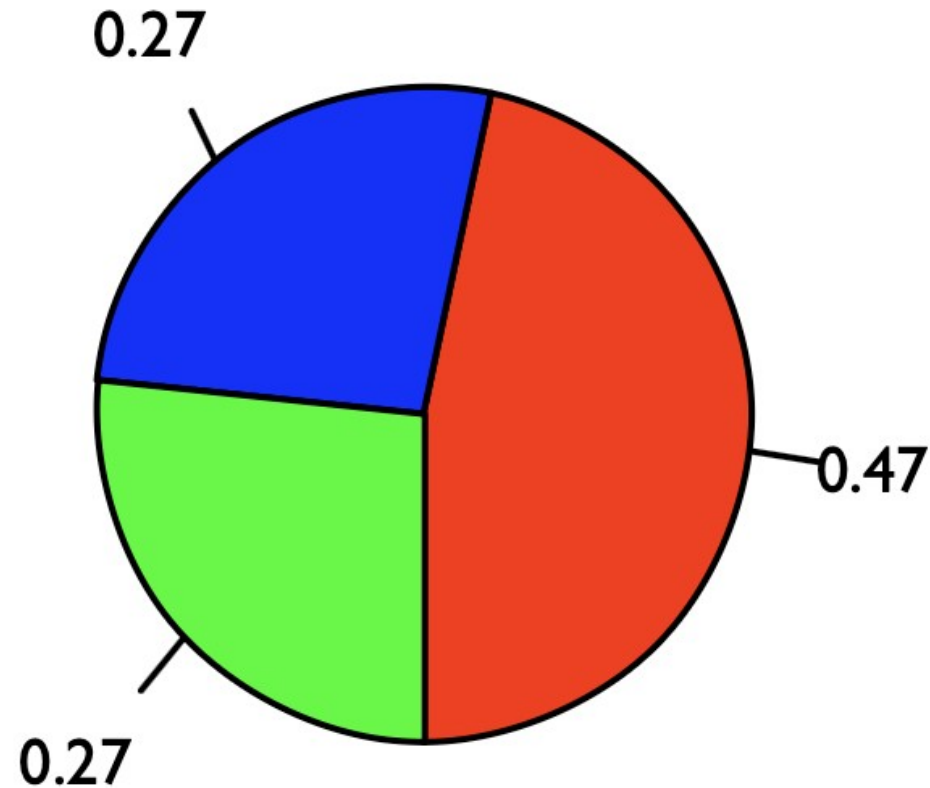
E-Step: Update the Classification of Reads



M-Step: Estimate the Abundances of Transcripts

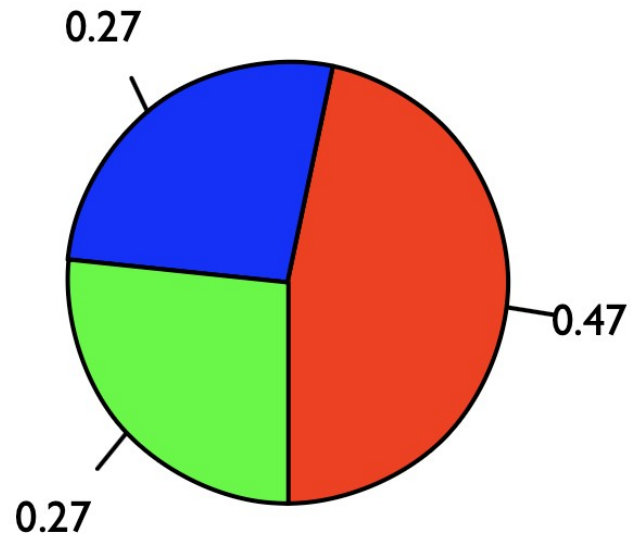


M-Step results in an **Updated Transcript Abundances**

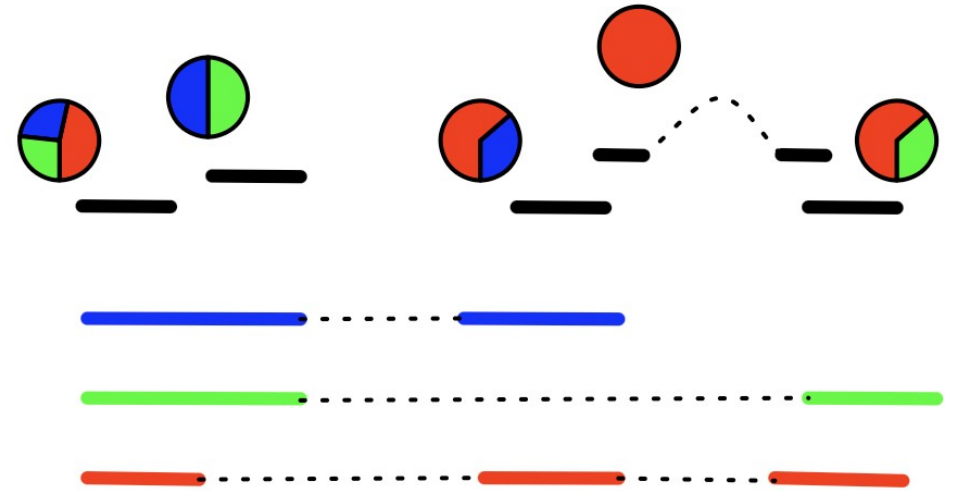


E-Step: Update the Classification of

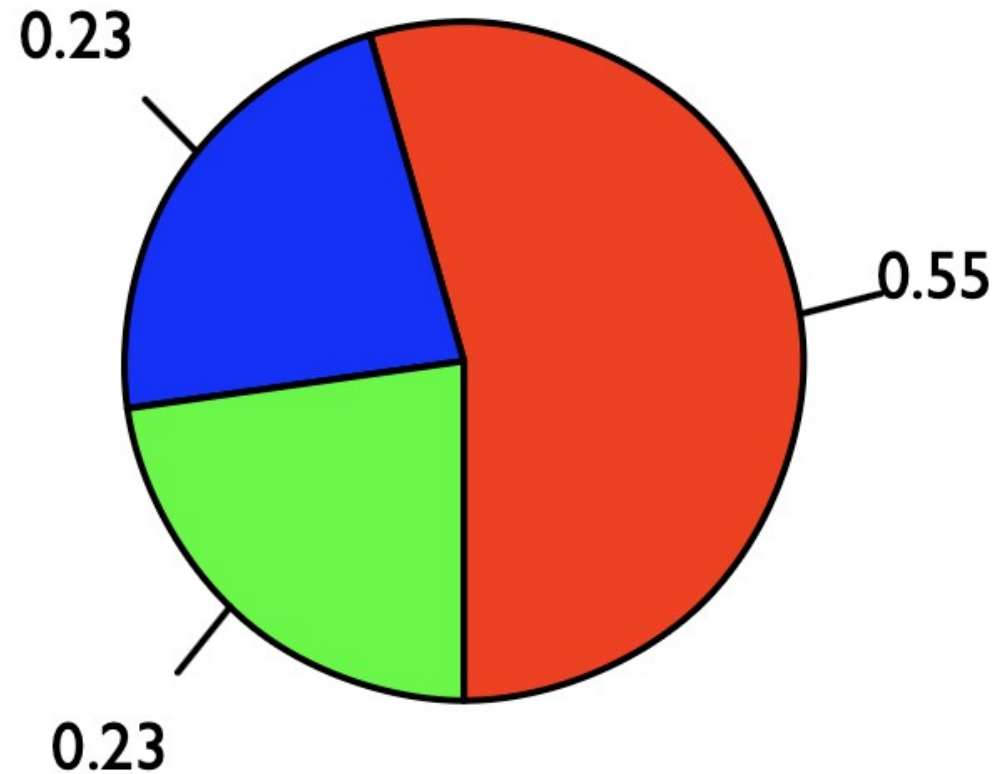
Nodes

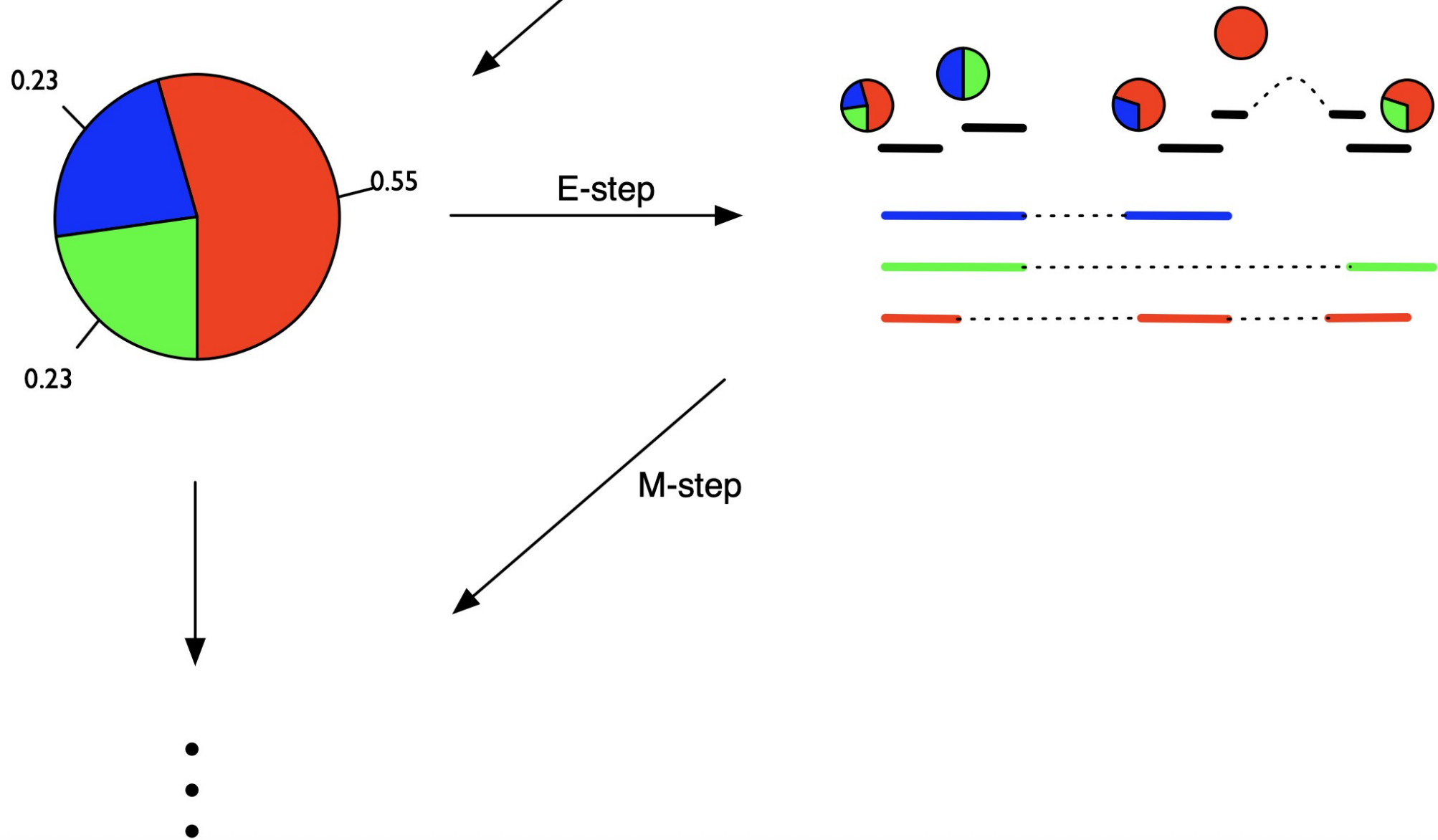


E-step



M-Step results in an **Updated Transcript Abundances**





Do E-Step and M-Step iteratively, until transcript abundances converge

Output of EM-based Software for RNA-seq

(1) Isoform (Transcript) Expression Levels

- Unit is TMP or RPKM or FPKM

(2) Gene expression levels

- e.g. Gene A = IsoformA_1 + IsoformA_2 +

The same EM in ten lines

Two transcripts. 6 reads unique to A, 2 unique to B, 4 compatible with both.

Python

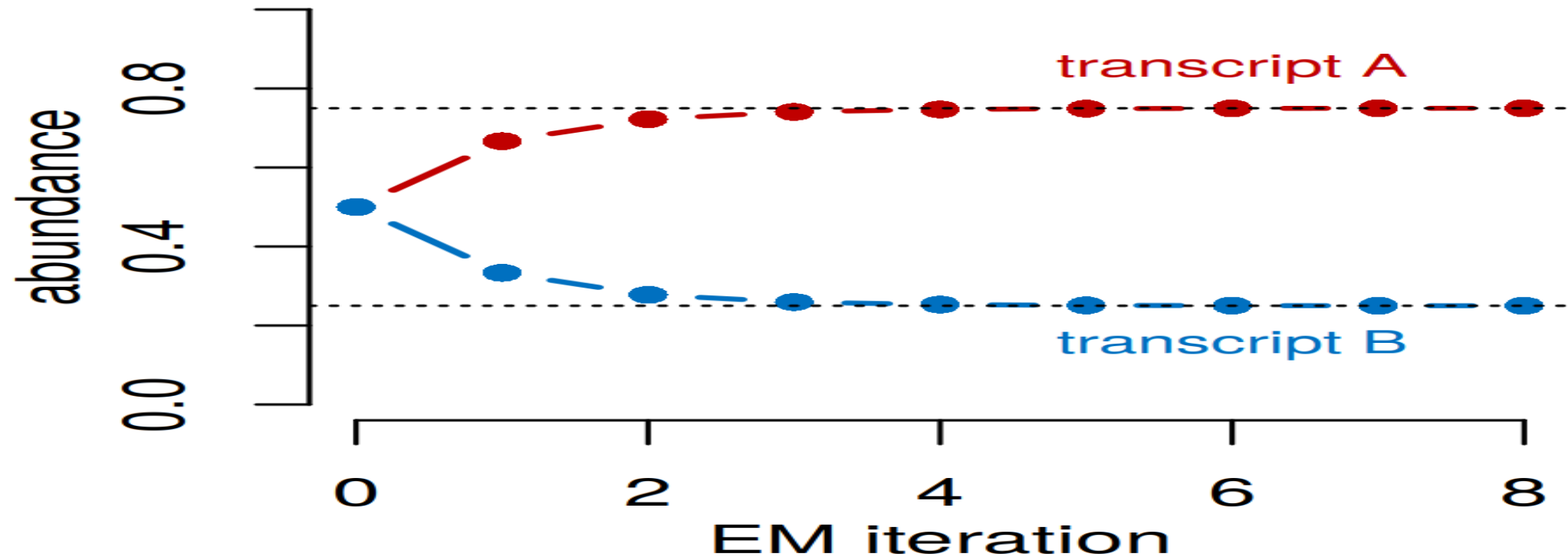
```
nA, nB, nShared = 6, 2, 4
theta = [0.5, 0.5]          # initiation
for it in range(8):
    # E-step: split shared reads
    eA = nA + nShared * theta[0] / sum(theta)
    eB = nB + nShared * theta[1] / sum(theta)
    # M-step: new abundances
    theta = [eA / (eA + eB), eB / (eA + eB)]
    print(it + 1, [round(t, 3) for t in theta])
# 1 [0.667, 0.333] ... 6 [0.75, 0.25]
```

R

```
nA <- 6; nB <- 2; nShared <- 4
theta <- c(0.5, 0.5)        # initiation
for (it in 1:8) {
    # E-step: split shared reads
    eA <- nA + nShared * theta[1] / sum(theta)
    eB <- nB + nShared * theta[2] / sum(theta)
    # M-step: new abundances
    theta <- c(eA, eB) / (eA + eB)
    cat(it, round(theta, 3), "\n")
}
# 1 0.667 0.333 ... 6 0.75 0.25
```

Naive 50/50 splitting gives $A = 0.667$. EM converges to 0.75: the more abundant transcript claims more of the shared reads, self-consistently.

Convergence in six iterations



Dotted lines: the fixed point 0.75 / 0.25. Each E-M round moves toward it and never overshoots.

RSEM does this for every isoform of every gene with every multi-mapping read — millions of hidden labels at once.

RSEM output is the E-step's last assignment

- Your BIOL419 Lab 4 file `sample_rsem.genes.results` — now `chr22_expression.tsv` on the course site.
- **expected_count = 317.84 for IGLC2**: a fractional count, because shared reads were split by posterior probability, not by coin flip.
- TPM then normalizes expected counts by effective length and to one million — the L7 formula.
- **ACTBP15, a pseudogene, has 650 expected reads**: with only chr22 in the reference, β -actin reads had nowhere else to go. EM cannot fix a missing reference.

EM resolves ambiguity among the choices it is given. Give it the whole genome, or it will confidently assign reads to the wrong place.

Part 4

Differential expression: DESeq2

Size factors, a negative-binomial GLM, dispersion shrinkage, and the multiple-testing correction.

Step 1 — Size factors (median of ratios)

Libraries differ in depth (15.7 M to 31.8 M reads). Divide each sample by a factor — but not by total counts.

Python

```
lg = np.log(cts.replace(0, np.nan))
ref = lg.mean(axis=1)          # per-gene log
                                geometric mean
sf = np.exp((lg.sub(ref, axis=0)).median())
sf.round(2)
# 1.02 0.90 1.18 0.67 1.17 1.39 0.92 0.95
```

R

```
lg <- log(as.matrix(cts)); lg[!is.finite(lg)] <-
NA
ref <- rowMeans(lg, na.rm = TRUE)
sf <- exp(apply(lg - ref, 2, median, na.rm =
TRUE))
round(sf, 2)
# 1.02 0.90 1.18 0.67 1.17 1.39 0.92 0.95
```

Median of ratios ignores the few giant genes that dominate totals. A sample's size factor is 'how much deeper than a typical gene's reference'.

Step 2 — The model: NB GLM

- **Count_gene,sample** ~ NB(μ , α_{gene}), $\log \mu = \log(\text{size factor}) + \beta_0 + \beta_{\text{cell}} \cdot \text{cell} + \beta_{\text{dex}} \cdot \text{dex}$
- The design ~ cell + dex says: each donor has its own baseline (pairing), and dex shifts every donor by the same log fold change β_{dex} .
- β_{dex} is the log2 fold change you report. Testing $\beta_{\text{dex}} = 0$ is the differential-expression test.
- This is the same formula syntax as `boxplot(y ~ group)` and `lm(y ~ x)` — R's formula is the interface to all of it.

A t-test compares two piles of numbers. A GLM fits a model of how the counts arose — depth, donor, drug.

Step 3 — Dispersion shrinkage: borrowing strength

- With 4 vs 4, each gene's α estimate is terrible: some genes look perfectly consistent by luck and would produce tiny p-values.
- DESeq2 fits a trend of α against mean across all 19,271 genes, then pulls each gene's α toward the trend (empirical Bayes).
- Genes with more information move less; genes with less information borrow more.
- This single idea — share information across genes — is what limma (2004), edgeR (2010) and DESeq2 (2014) have in common.

You have 4 replicates per gene but 19,271 genes per replicate. Use them.

Step 4 — Test, then correct

- **Wald test:** $\beta_{\text{dex}} / \text{SE}(\beta_{\text{dex}})$ compared with a normal — one p-value per gene.
- **Benjamini-Hochberg:** sort the 19,271 p-values; the k-th smallest is called at FDR q if $p_{(k)} \leq q \cdot k/n$.
- FDR 0.1 means: among the genes you call, about 10 % are expected to be false — a statement about the list, not about one gene.
- **lfcShrink:** shrink log fold changes of low-count genes so a 3-vs-0 gene does not top the ranking.

BH by hand — and the price of ignoring it

Python

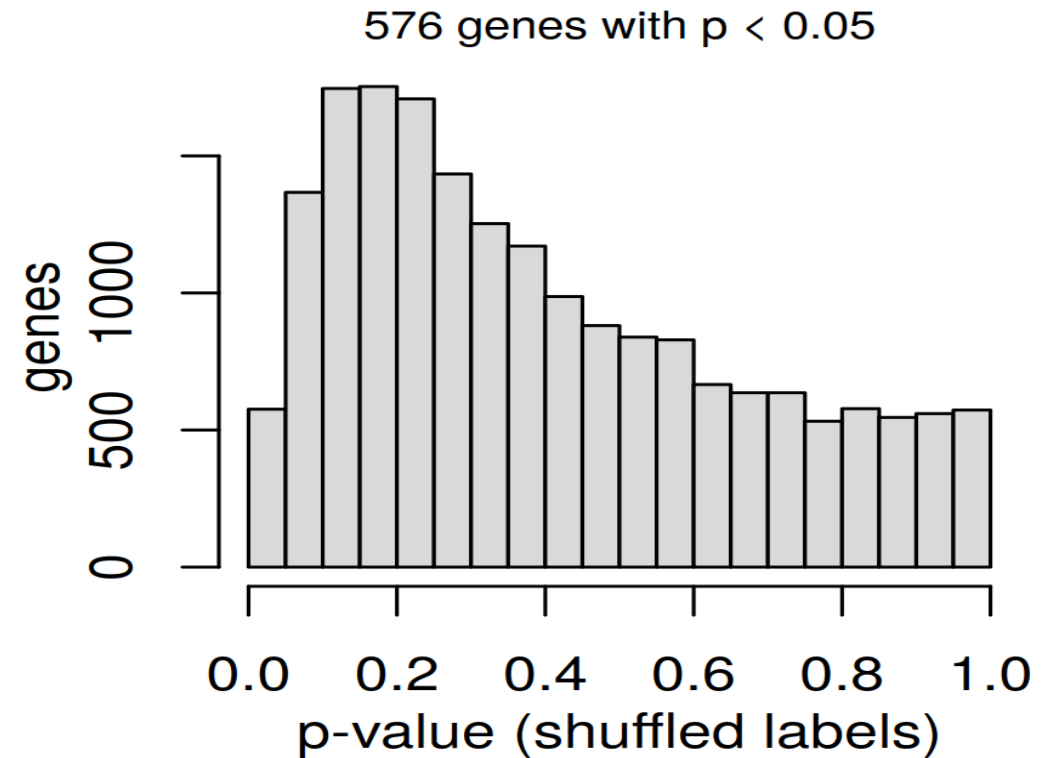
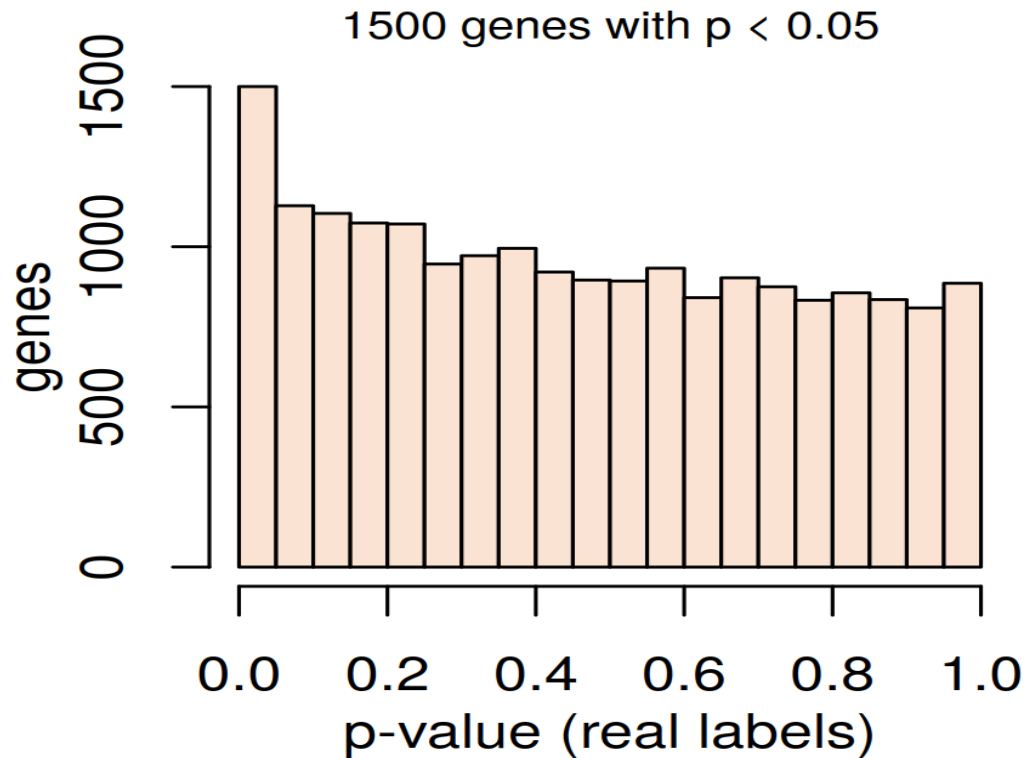
```
p = stats.ttest_ind(logc[trt], logc[ctrl],
                    axis=1).pvalue
def bh(p):
    p = np.asarray(p); n = len(p)
    o = np.argsort(p)
    r = p[o] * n / np.arange(1, n + 1)
    adj = np.minimum.accumulate(r[::-1])[::-1]
    q = np.empty(n); q[o] = np.minimum(adj, 1)
    return q
q = bh(p)
(p < 0.05).sum()      # 1812
(q < 0.05).sum()      # 41
```

R

```
tt <- function(a, b) tryCatch(
  t.test(a, b, var.equal = TRUE)$p.value,
  error = function(e) NA)
p <- apply(logc, 1,
           function(x) tt(x[trt], x[ctrl]))
q <- p.adjust(p, method = "BH")
sum(p < 0.05, na.rm = TRUE)  # 1812
sum(q < 0.05, na.rm = TRUE)  # 41
```

1,812 genes at $p < 0.05 \rightarrow 41$ at FDR 0.05. The other 1,771 were mostly what 5 % of 19,271 looks like.

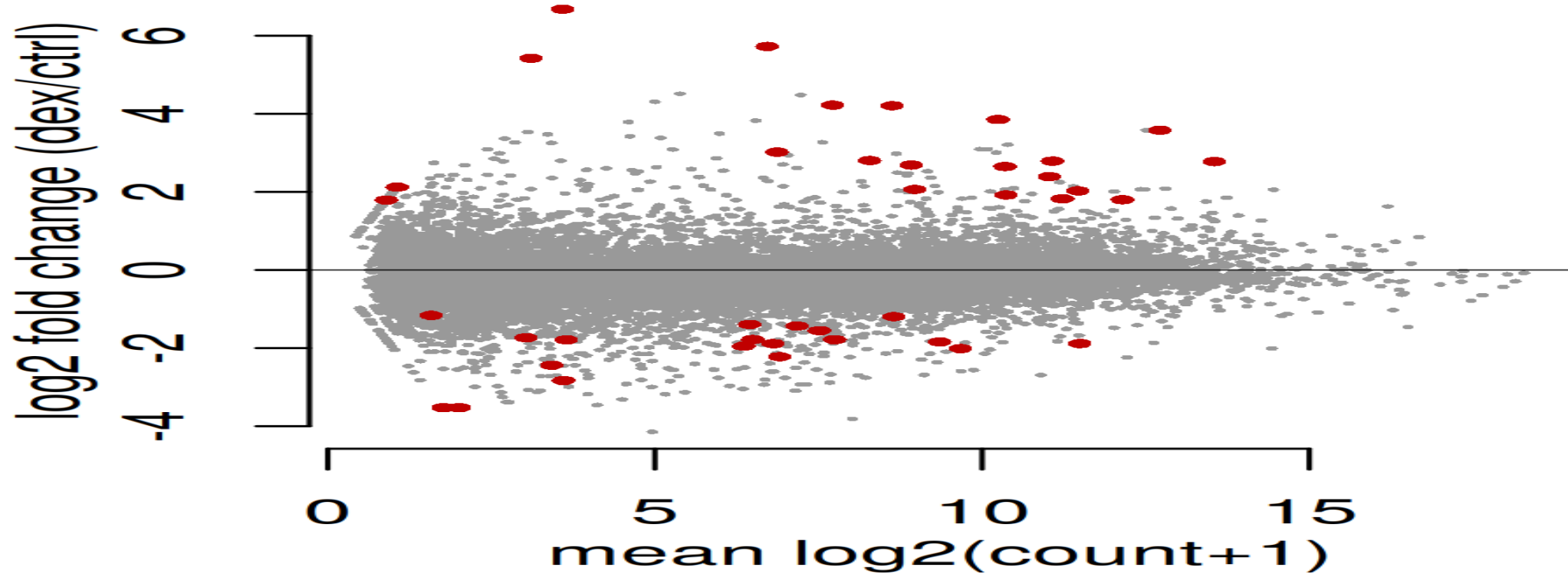
p-values with real labels vs shuffled labels



Left: dex vs control. Right: the same test after shuffling which sample is which.

Under the null, p-values are uniform — a flat histogram. The spike near 0 on the left is the signal; its height above the flat floor is roughly the number of real discoveries.

MA plot: look at every result this way



x: mean expression; y: log2 fold change. Red: the 41 genes surviving BH with a t-test.

Is the change spread across expression levels (good) or piled up at low counts (suspicious)? One picture answers it.

What DESeq2 finds on the same data

- The t-test keeps ~40 genes after BH. DESeq2 with design ~ cell + dex reports several thousand at FDR 0.1 (DESeq2 vignette, airway).
- Not because DESeq2 is lenient: the NB model, the shrunken dispersion and the paired design have far more power than an unpaired t-test on log counts.
- **Three lines on your laptop:** `dds <- DESeqDataSetFromMatrix(cts, meta, ~ cell + dex);`
`dds <- DESeq(dds); res <- results(dds)`
- Bioconductor only — it does not run in the browser. The Setup page has the script.

The gap between 40 and thousands is the whole reason to learn the model.

Reading a DESeq2 result

YOUR TURN A gene has $\log_2\text{FoldChange} = 4.1$, $\text{baseMean} = 3$, $\text{padj} = 0.04$. Do you put it in the paper's headline figure?

- A. Yes — $\text{padj} < 0.05$ and a 17-fold change
- B. No — $\text{baseMean} = 3$ means a handful of reads; the fold change is unstable
- C. Only after `lfcShrink` and a look at the individual counts

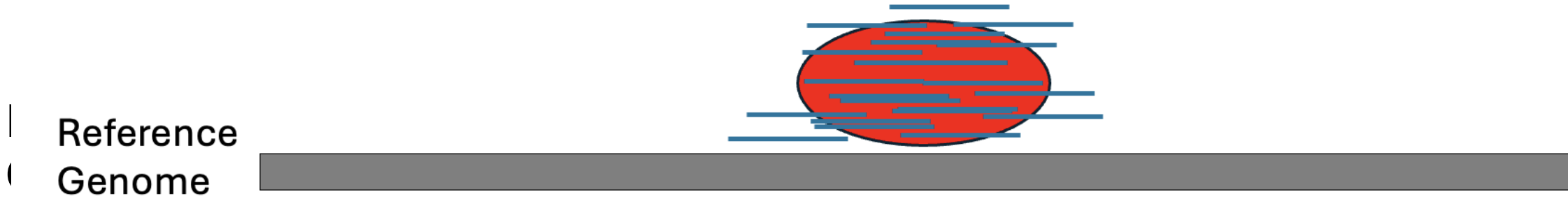
ANSWER C. Significance says the change is unlikely to be zero; it does not say the size is reliable. Low counts → shrink, then look.

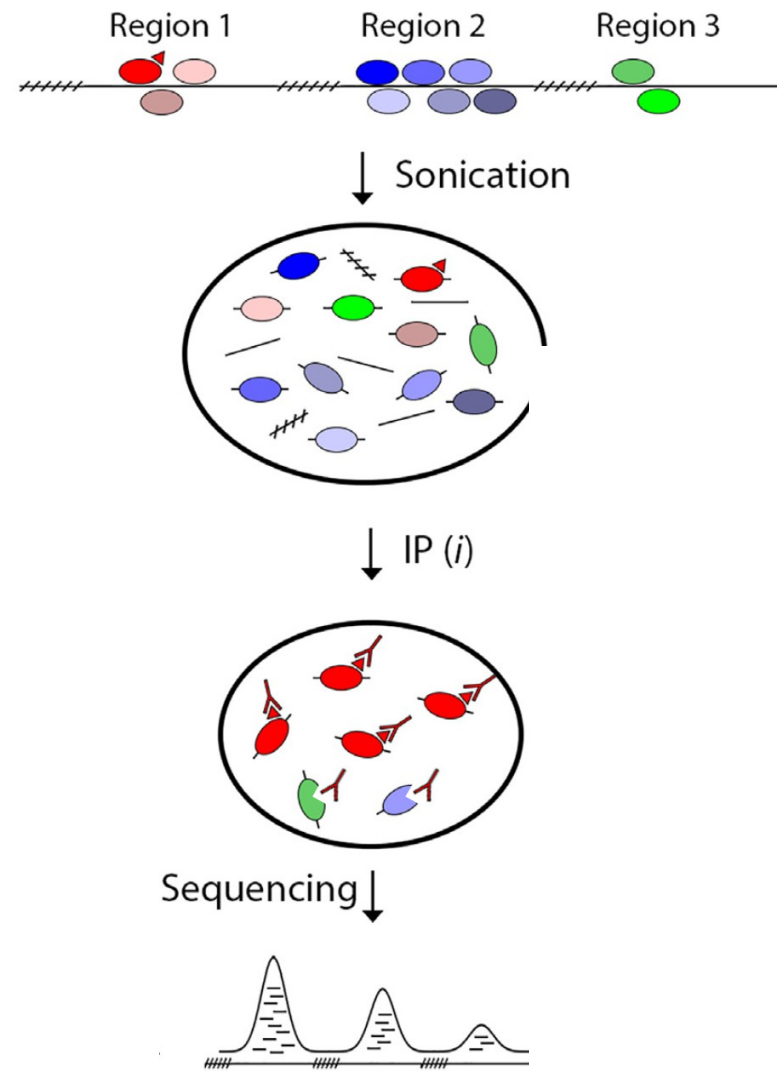
Part 5

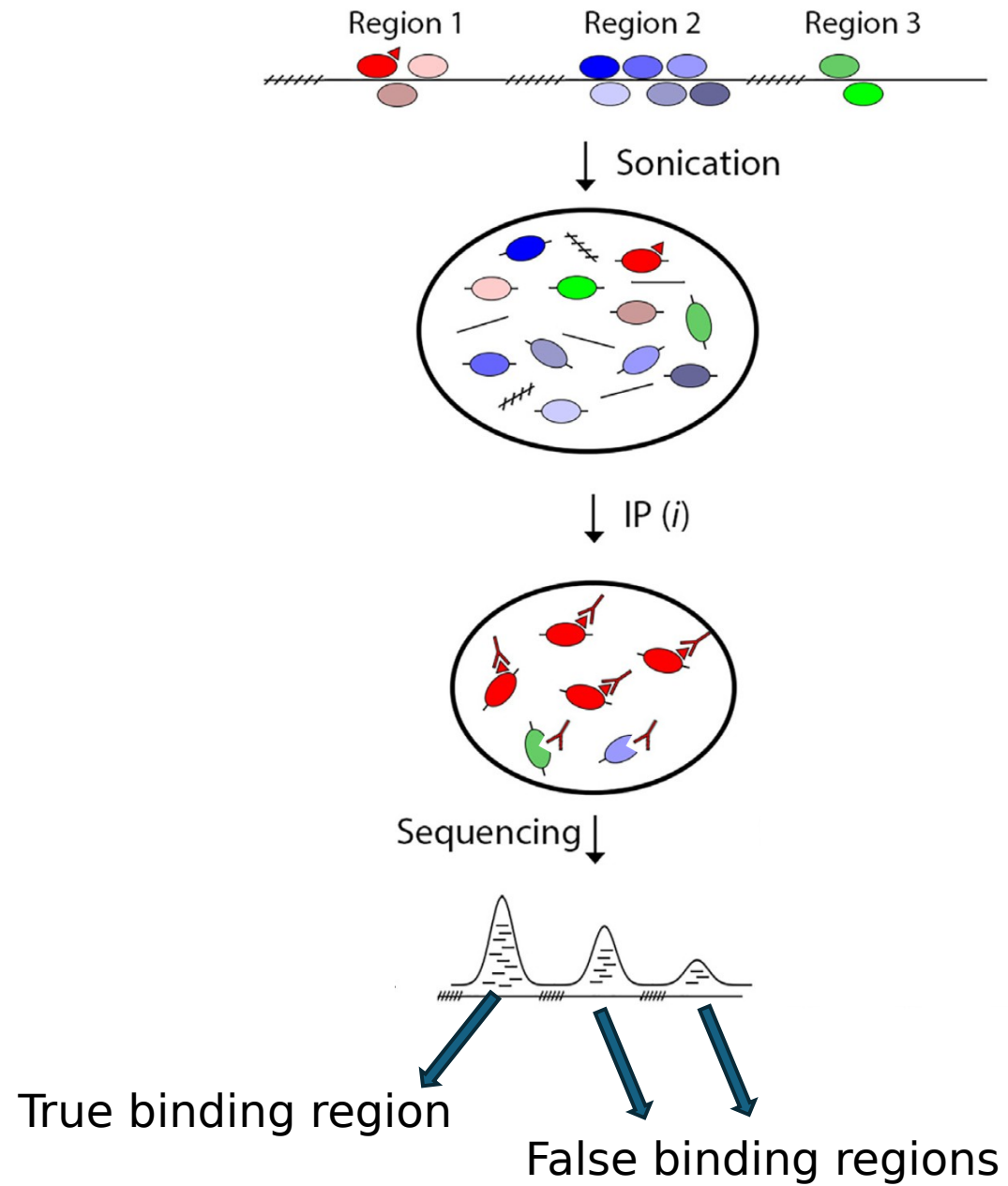
ChIP-seq: designing the null

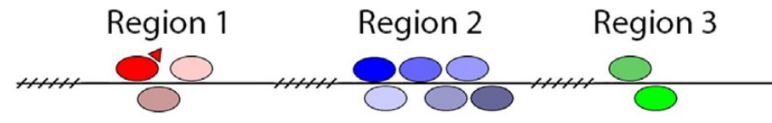
Peaks are Poisson tests against a control — so which control? From BIOL419 Lecture 13 and Xu et al., NAR 2021.

Chromatin Immuno-Precipitation followed by Sequencing (**ChIP-Seq**)

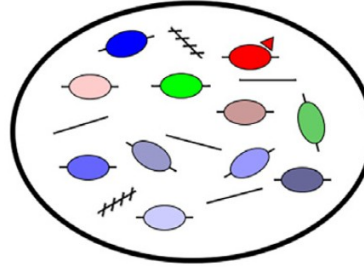




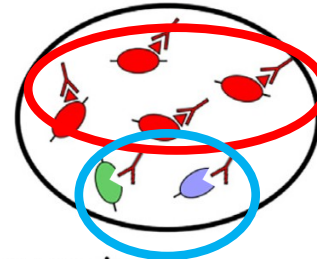




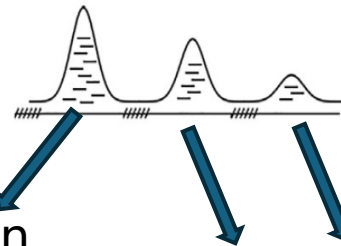
↓ Sonication



↓ IP (i)



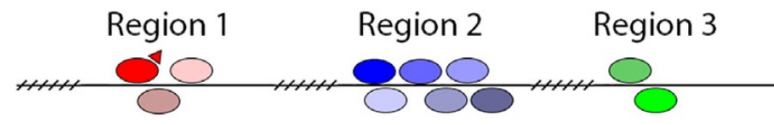
Sequencing ↓



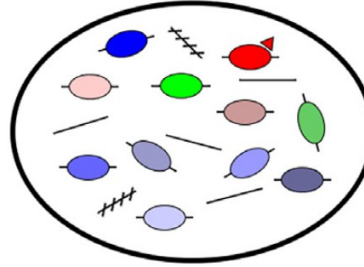
True binding region

False binding regions

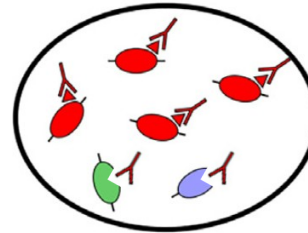
ChIP-seq signals =
Specific binding between
 antibody and **target TF**
 + **Nonspecific** binding
 between antibody and
 other TFs



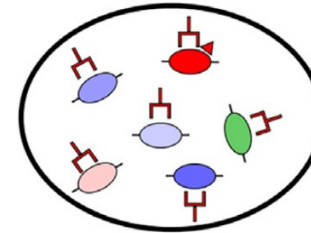
↓ Sonication



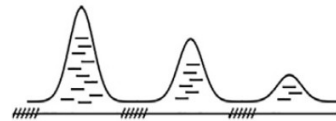
↓ IP (*i*)



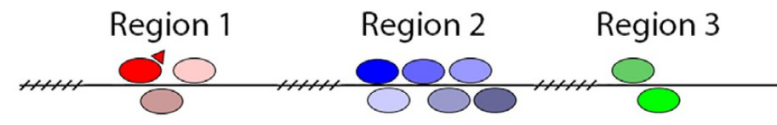
↘ Mock IP (*m*)



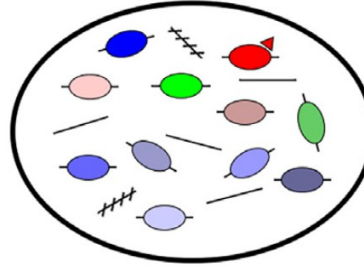
Sequencing ↓



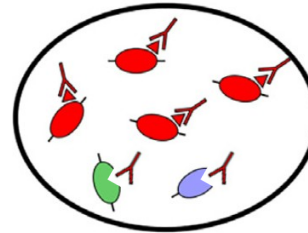
Use control
experiment to
mimic nonspecific
binding



↓ Sonication

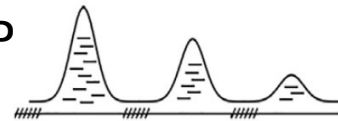


↓ IP (*i*)

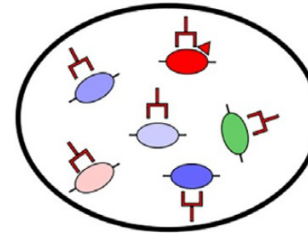


Sequencing ↓

Profile IP

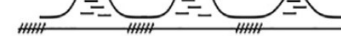


↘ Mock IP (*m*)

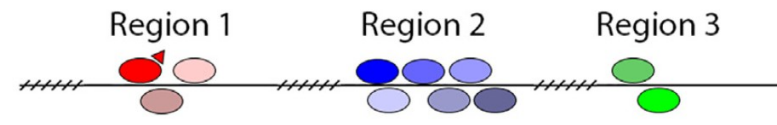


Sequencing ↓

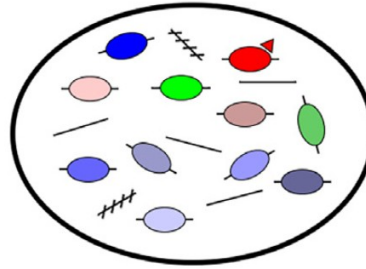
Profile Mock IP



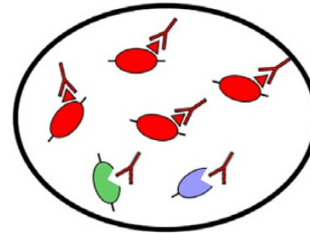
Use control experiment
to mimic nonspecific
binding



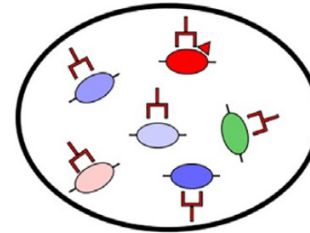
↓ Sonication



↓ IP (*i*)



↘ Mock IP (*m*)

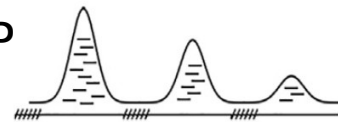


Use control experiment to mimic nonspecific binding

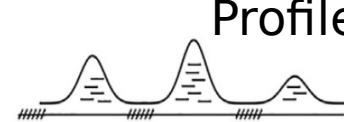
Sequencing ↓

Sequencing ↓

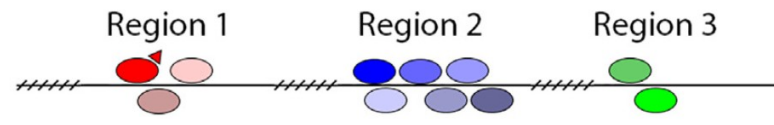
Profile IP



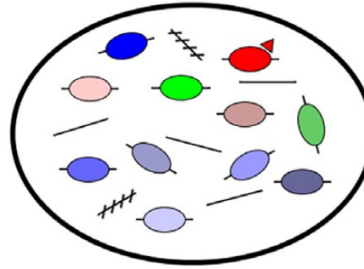
Profile Mock IP



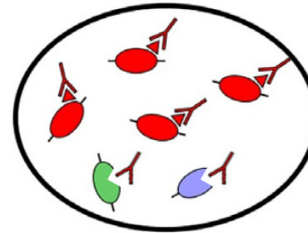
Profile of IP minus Profile of Mock IP =



↓ Sonication

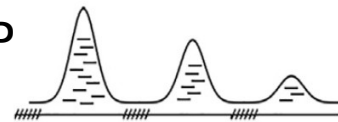


↓ IP (*i*)

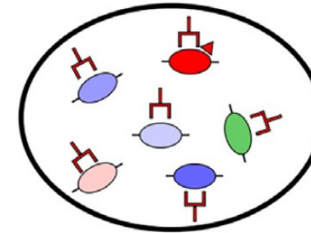


Sequencing ↓

Profile IP

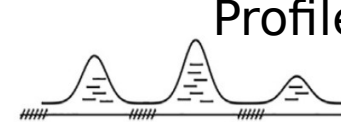


↘ Mock IP (*m*)

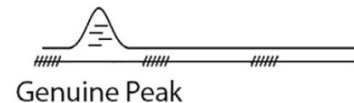


Sequencing ↓

Profile Mock IP

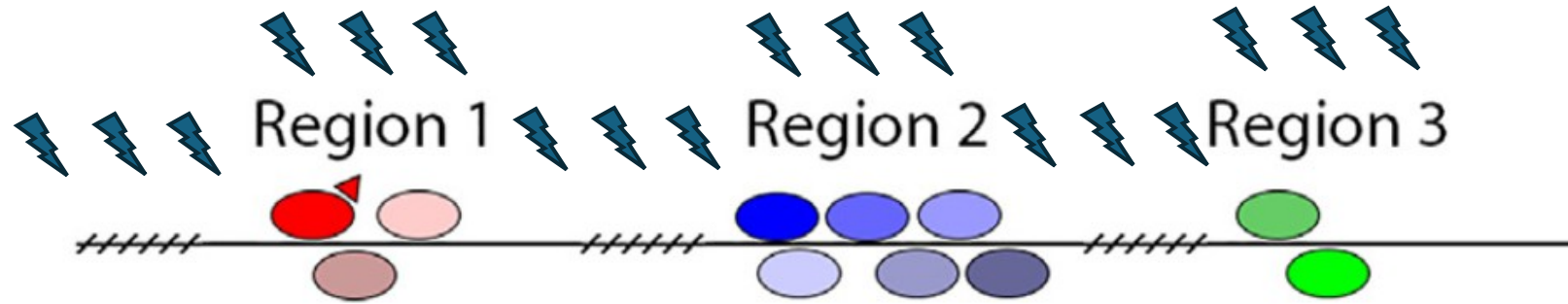


Peak Caller ↘ ✓



Profile of IP minus Profile of Mock IP =

Sonication Bias



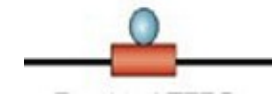
//// Closed Chromatin

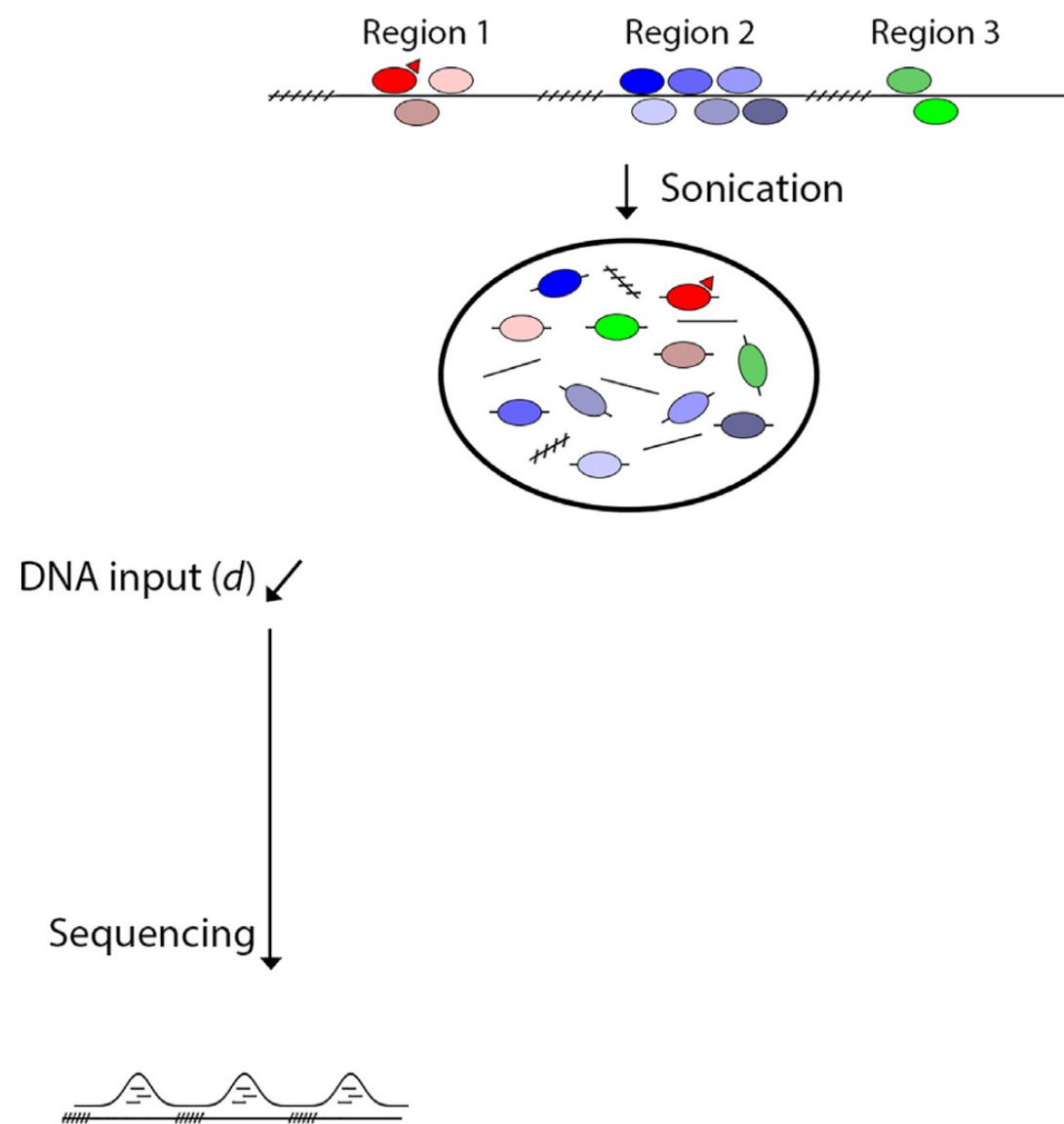
DNA densely packed with histones

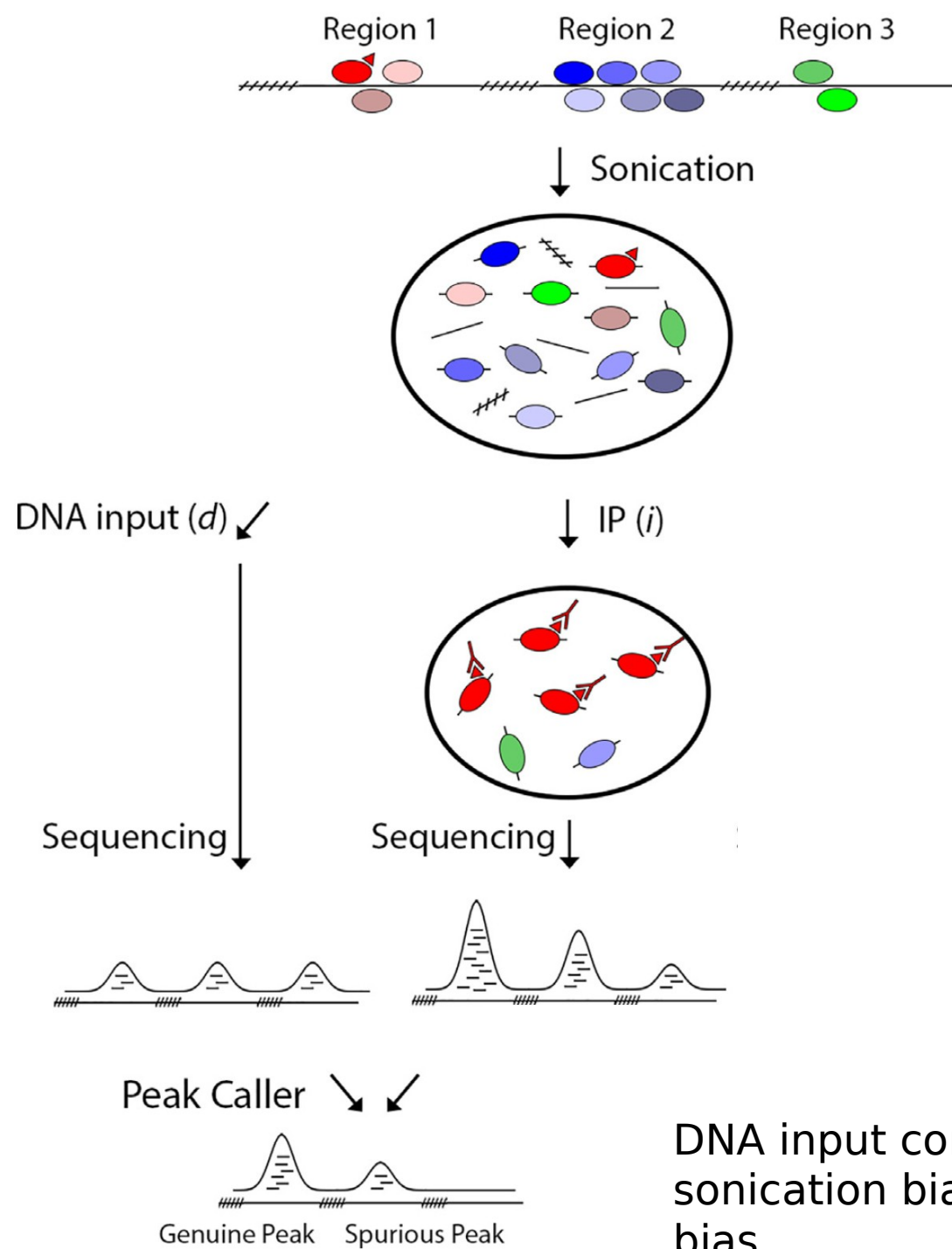


— Open Chromatin

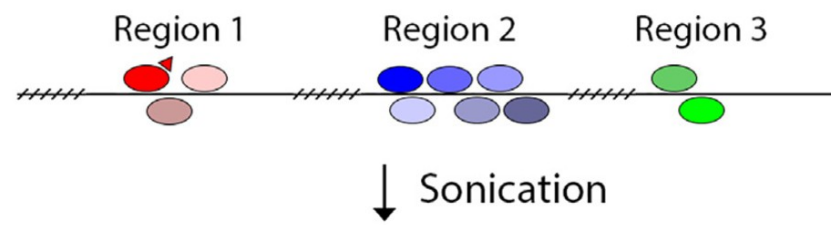
Naked DNA





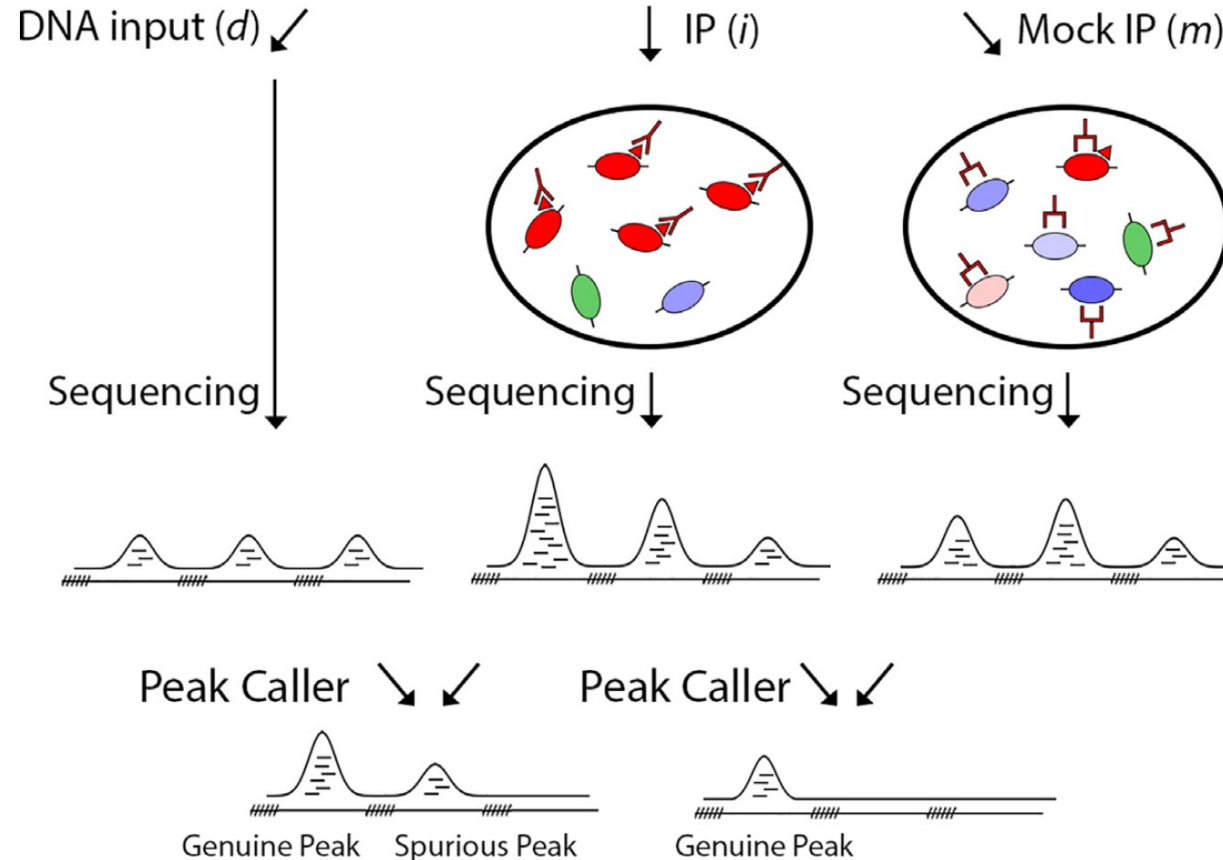


DNA input corrects only
sonication bias but binding
bias

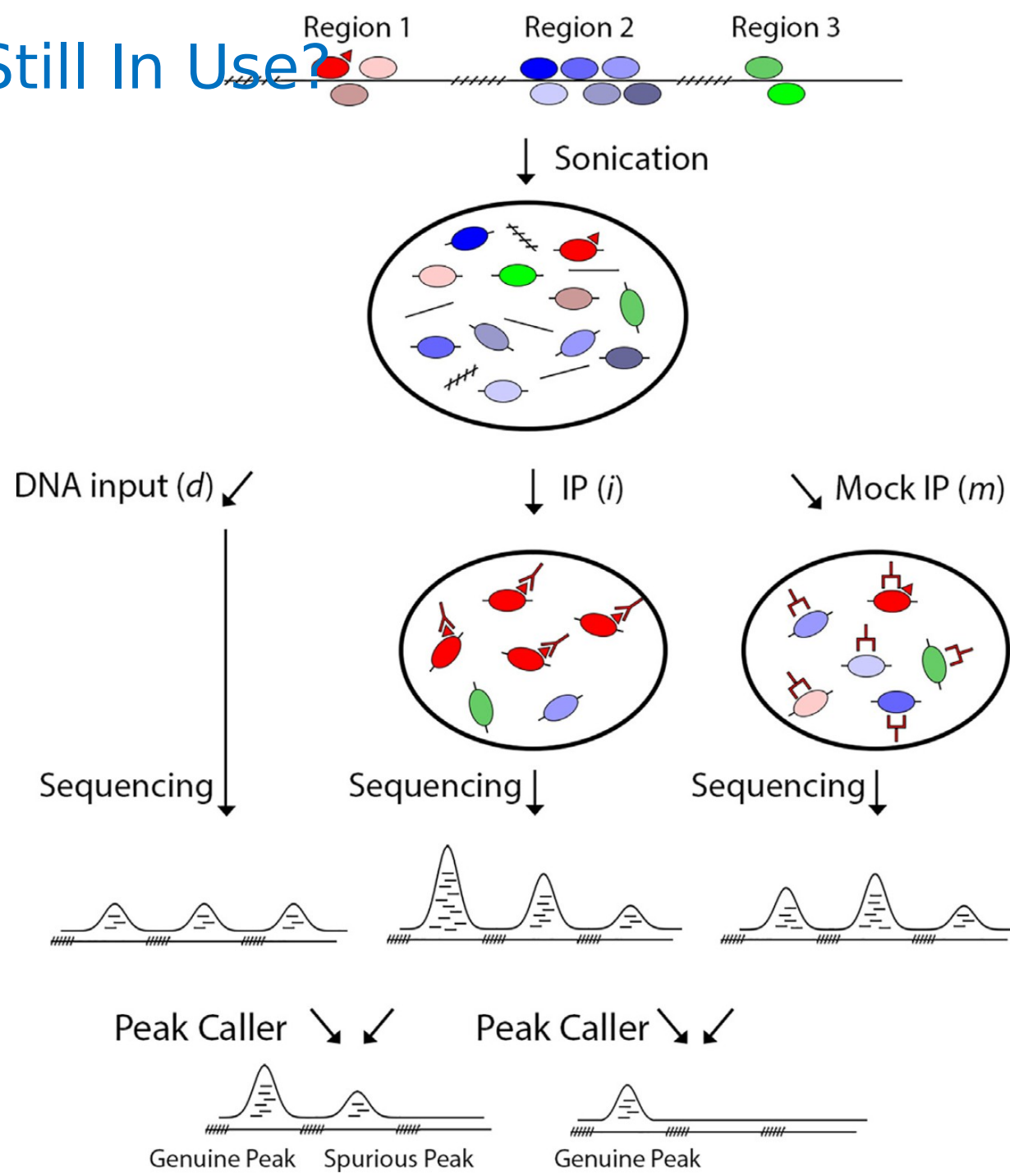


DNA input controls only
sonication bias

Mock IP controls both
non-specific interaction
and sonication bias



Why DNA Input Still In Use?



Maximizing the Advantage of both DNA input and Mock IP

Published online 21 December 2020

Nucleic Acids Research, 2021, Vol. 49, No. 3 e17
doi: 10.1093/nar/gkaa1155

To mock or not: a comprehensive comparison of mock IP and DNA input for ChIP-seq

Jinrui Xu^{1,2}, Michelle M. Kudron^{3,†}, Alec Victorsen^{4,†}, Jiahao Gao^{1,2,†}, Haneen N. Ammouri⁴, Fabio C. P. Navarro^{1,2}, Louis Gevirtzman⁵, Robert H. Waterston⁵, Kevin P. White⁴, Valerie Reinke³ and Mark Gerstein^{1,2,6,7,*}

¹Program in Computational Biology and Bioinformatics, Yale University, New Haven, CT 06520, USA, ²Department of Molecular Biophysics and Biochemistry, Yale University, New Haven, CT 06520, USA, ³Department of Genetics, Yale University, New Haven, CT 06520, USA, ⁴Institute for Genomics and Systems Biology, Department of Human Genetics, University of Chicago, IL 60637, USA, ⁵Department of Genome Sciences, University of Washington School of Medicine, Seattle, WA 98195, USA, ⁶Department of Computer Science, Yale University, New Haven, CT 06520, USA and ⁷Department of Statistics and Data Science, Yale University, New Haven, CT 06520, USA

$$P(\theta_1 > 0.5 \ \& \ \theta_1 > \theta_2) = \int_{\theta_2=0}^{\theta_1} \int_{\theta_1=0.5}^1 \text{Beta}(\theta_1 \mid n_i + 1, n_d + 1) \\ \text{Beta}(\theta_2 \mid n_{m'} + 1, n_{d'} + 1) d\theta_1 d\theta_2$$

King the Advantage of both DNA input and Mock IP

$$P(\theta_1 > 0.5 \ \& \ \theta_1 > \theta_2) = \int_{\theta_2=0}^{\theta_1} \int_{\theta_1=0.5}^1 \text{Beta}(\theta_1 \mid n_i + 1, n_d + 1) \\ \text{Beta}(\theta_2 \mid n_{m'} + 1, n_{d'} + 1) d\theta_1 d\theta_2$$

Maximizing the Advantage of both DNA input and Mock IP

$$P(\theta_1 > 0.5 \ \& \ \theta_1 > \theta_2) = \int_{\theta_2=0}^{\theta_1} \int_{\theta_1=0.5}^1 \text{Beta}(\theta_1 \mid n_i + 1, n_d + 1) \\ \text{Beta}(\theta_2 \mid n_{m'} + 1, n_{d'} + 1) d\theta_1 d\theta_2$$

To mock or not: what the data said (Xu et al., NAR 2021)

- modERN datasets with matched IP, DNA input and mock IP: 113 human (6 cell lines), 317 worm, 182 fly experiments.
- **Mock IP removes far more spurious sites than input:** ~10 % of input-called peaks in human cell lines, ~60 % in worm and ~88 % in fly (whole animals, many cell types).
- **Peaks surviving mock IP are more often real:** motif enrichment in top peaks rises 18–37 %.
- But mock IP libraries are shallow and noisy (little DNA); alone they lose sensitivity.
- **Recommendation: use both.** Call against input for sensitivity, then score each peak by IP-vs-mock with a Bayesian posterior.

The posterior, in words

- For a region: n_i reads in IP, n_d in its input $\rightarrow \theta_1 = \text{IP fraction} \sim \text{Beta}(n_i + 1, n_d + 1)$.
- For the mock experiment: n_m in mock IP, $n_{d'}$ in the mock's input $\rightarrow \theta_2 \sim \text{Beta}(n_m + 1, n_{d'} + 1)$.
- **A genuine peak needs $\theta_1 > 0.5$ (enriched over input) and $\theta_1 > \theta_2$ (more enriched than the antibody-free control).**
- $P(\theta_1 > 0.5 \ \& \ \theta_1 > \theta_2 \mid \text{data})$ — the double integral on the previous slide. Rank peaks by it, combine with the peak caller's score, run IDR.

Same Beta posterior as Part 2, applied twice and compared. Nothing new was needed — only the right question.

The posterior by simulation

Draw θ_1 and θ_2 from their Beta posteriors 100,000 times; count how often the peak condition holds.

Python

```
rng = np.random.default_rng(0)
def posterior(n_i, n_d, n_m, n_dm, N=100000):
    t1 = rng.beta(n_i + 1, n_d + 1, N)
    t2 = rng.beta(n_m + 1, n_dm + 1, N)
    return ((t1 > 0.5) & (t1 > t2)).mean()

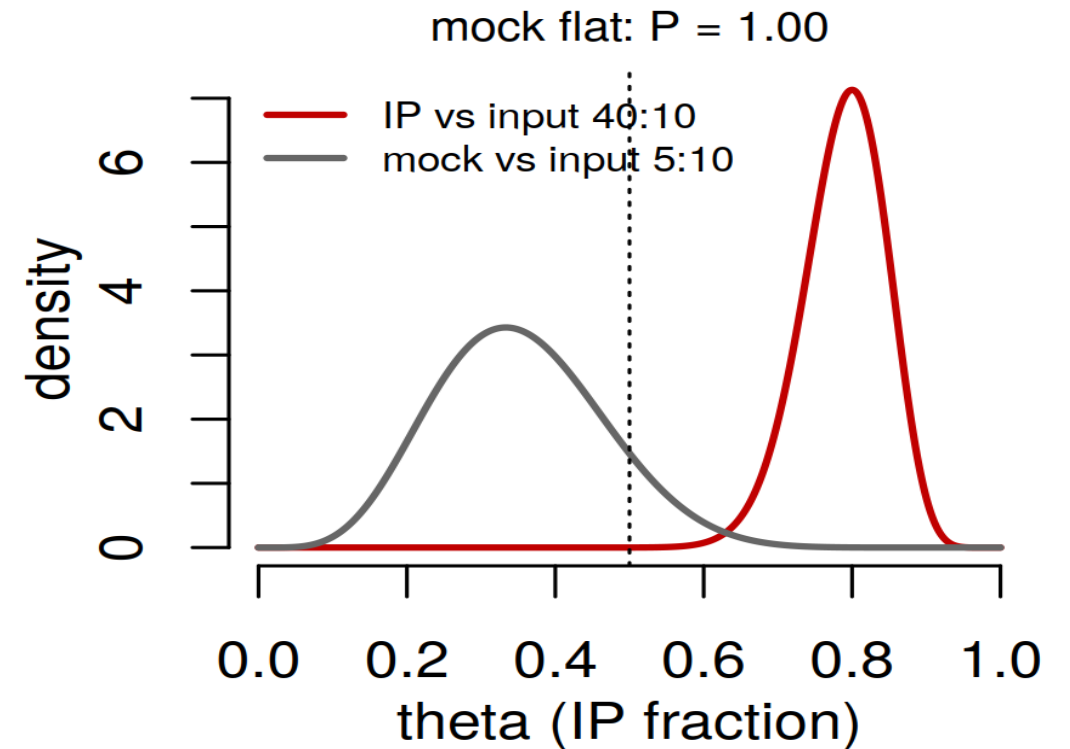
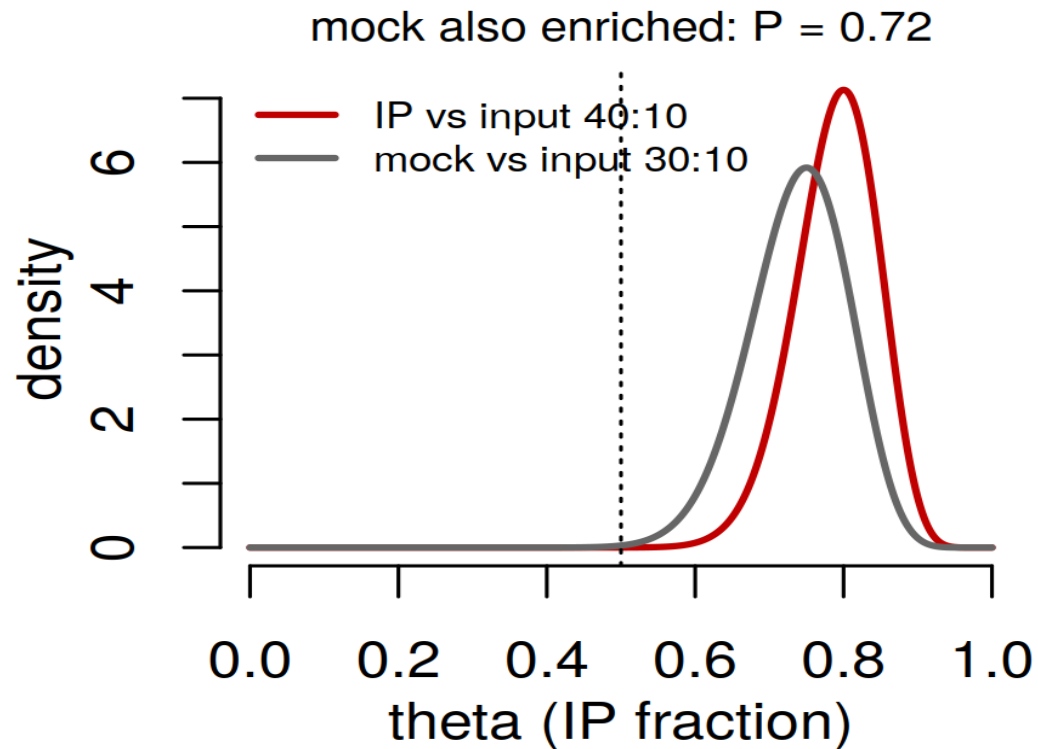
posterior(40, 10, 30, 10)    # 0.72
posterior(40, 10, 5, 10)    # 1.00
```

R

```
set.seed(0)
posterior <- function(n_i, n_d, n_m, n_dm,
                      N = 100000) {
  t1 <- rbeta(N, n_i + 1, n_d + 1)
  t2 <- rbeta(N, n_m + 1, n_dm + 1)
  mean(t1 > 0.5 & t1 > t2)
}
posterior(40, 10, 30, 10)    # 0.72
posterior(40, 10, 5, 10)    # 1.00
```

Same IP enrichment (40 vs 10) in both cases. What changes the verdict is the mock: enriched → 'hyper-ChIPable' region; flat → real peak.

Two peaks with identical IP signal



Red: IP-vs-input posterior. Grey: mock-vs-input posterior. Left: the mock is also enriched. Right: the mock is flat.

DNA input alone would call both. Mock IP alone would miss weak real peaks. Together they separate the two cases.

Design the control

YOUR TURN You are doing ChIP-seq for a low-abundance TF in whole fly embryos. Which control, and why?

- A. DNA input only — cheapest, deepest library
- B. Mock IP only — removes non-specific binding
- C. Both: input for calling, mock for scoring

ANSWER C. In fly, 88 % of input-only peaks were spurious; but a mock library alone is too shallow for a low-abundance TF. Use input for sensitivity, mock to remove hyper-ChIPable regions.

Part 6

The toolbox

One table to carry into every genomics paper you read.

Which tool for which question?

You have...	Ask...	Tool
k of n reads carry an allele	is $\theta = 0.5$? what is $P(\theta > 0.5)$?	binomial test · Beta posterior
reads in a window vs background	more than chance?	Poisson tail test (MACS)
counts × replicates × conditions	which genes change?	NB GLM + shrinkage + BH (DESeq2, edgeR)
reads compatible with several isoforms	how much of each?	EM (RSEM, kallisto, salmon)
thousands of p-values	how many are false?	Benjamini–Hochberg FDR
a peak in IP	is it in mock too?	joint Beta posterior (Xu et al. 2021)

What you can now do

- Say which distribution a genomic count follows, and why replicates need the negative binomial.
- Explain likelihood, a Beta posterior, and the E- and M-steps — and run EM in ten lines.
- Read a DESeq2 result: size factor, design, dispersion, Wald p, BH padj, shrunken LFC.
- Explain why a peak needs a control, and what mock IP adds to DNA input.

Tomorrow's lab: binomial test on real mismatches, the 19,271-tests trap, BH by hand.

References

- Love MI, Huber W, Anders S. Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. Genome Biol 2014.
- Li B, Dewey CN. RSEM: accurate transcript quantification from RNA-Seq data. BMC Bioinformatics 2011.
- Zhang Y et al. Model-based analysis of ChIP-Seq (MACS). Genome Biol 2008.
- Xu J, Kudron MM, Victorsen A, ... Reinke V, Gerstein M. To mock or not: a comprehensive comparison of mock IP and DNA input for ChIP-seq. Nucleic Acids Res 2021;49(3):e17.
- Himes BE et al. RNA-Seq transcriptome profiling identifies CRISPLD2 as a glucocorticoid responsive gene in airway smooth muscle cells. PLoS ONE 2014.
- Benjamini Y, Hochberg Y. Controlling the false discovery rate. J R Stat Soc B 1995.

Questions?

jinrui.xu@howard.edu

Course site: Lecture 11 notes with every demo runnable in the browser.